

ỦY BAN NHÂN DÂN TP . HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP TUẦN

Học phần: Seminar Chuyên đề

Đề tài: Cơ sở lập trình với GenAI (Nâng cao)

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Lớp: DCT122C3

Nhóm sinh viên thực hiện:

STT	Họ và tên	MSSV
1	Nguyễn Hữu Anh Khoa	3122411098
2	Võ Thành Danh	3122411024
3	Trang Gia Huy	3122411068
4	Nguyễn Lê Quỳnh Hương	3122411078

MỤC LỤC

LỜI MỞ ĐẦU	11
CHƯƠNG 6: CƠ CHẾ HOẠT ĐỘNG CỦA LLM.....	12
6.1. Giới thiệu	12
6.2. Nguyên lý hoạt động của LLM	12
6.2.1. Bản chất của LLM	12
6.2.2. Từ mô hình thống kê đến AI	12
6.3. Kiến trúc tổng thể của LLM	13
6.3.1. Mô tả tổng quan	13
6.3.2 Diagram kiến trúc LLM (chi tiết).....	14
6.4. Tokenization (Phân tích chuyên sâu)	15
6.4.1 Token là gì?	15
6.4.2. Các phương pháp Tokenization	15
6.4.3 Tại sao Tokenization quan trọng?	16
6.4.4. Embedding.....	17
6.5. Attention (Phân tích chuyên sâu)	18
6.5.1 Vấn đề cần giải quyết	18
6.5.2 Cơ chế Attention.....	18
6.5.3 Multi-head Attention	19
6.6. Hạn chế của LLM.....	20
6.6.1. Thiếu sự hiểu biết thực sự về thế giới.....	20
6.6.2. Hiện tượng "Ảo giác" (Hallucination)	20
6.6.3. Sự phụ thuộc vào dữ liệu huấn luyện.....	20
6.6.4. Giới hạn về "Cửa sổ ngữ cảnh" (Context Window)	20
6.7. Kết luận	21
CHƯƠNG 7: ĐỌC VÀ HIỂU CODEBASES VỚI GENAI	22
7.1. Giới thiệu	22
7.2. Cấu trúc Codebase	22
7.3. Business Logic: Manhattan Distance	23
7.3.1 Khái niệm	23

7.3.2 Code minh họa	23
7.4. SỬ DỤNG GENAI ĐỂ HIỂU CODE.....	24
7.4.1. Cơ chế vận hành với lệnh /explain	24
7.4.2. Đánh giá hiệu quả ứng dụng	25
7.4.3. ChatGPT	25
7.4.3 OpenAI API	28
7.5. Hiểu non-project files	30
7.5.1 requirements.txt	30
7.5.2. Dockerfile	31
7.6. Debugging – Cách hiểu code hiệu quả nhất:	32
7.6.1. Quy trình thực nghiệm 3 bước.....	32
7.6.2. Giá trị của việc Debug trong phân tích hệ thống	32
7.7. Tạo dữ liệu mẫu bằng AI	33
7.7.1. Sử dụng GitHub Copilot để tự sinh mã nguồn (Auto-generation)	33
7.7.2. Tùy chỉnh dữ liệu chuyên sâu với ChatGPT	33
7.7.3. Lợi ích của việc kết hợp AI trong Testing	33
7.8. So sánh công cụ.....	34
7.9. Kết luận	34
7.9.1. Thay đổi tư duy tiếp cận mã nguồn	34
7.9.2. Vai trò chiến lược của Generative AI (GenAI)	34
7.9.3. Debugging: Kỹ năng định hình năng lực kỹ sư	34
7.9.4. Mô hình phân tích hiện đại	34
Chương 8: GIỚI THIỆU VỀ PROMPT ENGINEERING.....	36
8.1. Giới thiệu	36
8.2. Few-shot Learning.....	36
8.2.1 Khái niệm	36
8.2.2. Ứng dụng Few-shot Learning trong lập trình.....	36
8.3. Iterative Prompting	38
8.4 Template-based Prompting	40
8.5. Chain-of-Thought Prompting (CoT)	40
8.6 Kết luận	41

CHƯƠNG 9: KỸ THUẬT PROMPT NÂNG CAO CHO CÁC TÁC VỤ LIÊN QUAN ĐẾN MÃ NGUỒN	42
9.1 Giới thiệu	42
9.2. Mở rộng phạm vi thực thi của Prompt (Extending Prompt Scope)	42
9.3 Baseline Prompt và Kết quả ban đầu	42
9.4. Kỹ thuật Chain-of-Thought (CoT) trong lập trình	45
9.5 Sử dụng CoT (Chain of Thought) với GitHub Copilot	45
9.6. Áp dụng Chain of Thought (CoT) với ChatGPT và OpenAI API.....	47
9.6.1. CoT với ChatGPT	47
9.6.2. CoT với OpenAI API	48
9.7. Sử dụng Prompt Chaining với các mô hình LLM	50
9.7.1. Giới thiệu	50
9.7.2 Áp dụng Prompt Chaining với ChatGPT	50
9.7.3 Tối ưu Prompt Chaining với Selective History trong OpenAI API	52
9.7.4 Áp dụng Prompt Chaining với GitHub Copilot	55
9.8 Tổng kết	57
CHƯƠNG 10: REFACTORING CODE VỚI GENAI	58
10.1. Giới thiệu	58
10.2. Khái niệm Refactoring.....	58
10.2.1. Định nghĩa kỹ thuật	58
10.2.2. Các mục tiêu trọng tâm	58
10.2.3. Tầm quan trọng trong vòng đời phần mềm	58
10.3. Ví dụ minh họa: Hàm tính khoảng cách	59
10.3.1. Phân tích phiên bản mã nguồn ban đầu	59
10.3.2. Nhận diện "Code Smell" (Dấu hiệu mã nguồn xấu)	60
10.3.3. Định hướng Refactoring.....	60
10.4. Các loại Refactoring	61
10.4.1. Cải thiện khả năng đọc (Readability)	61
10.4.2. Thay đổi cấu trúc (Structure).....	61
10.4.3. Tối ưu hiệu năng (Performance).....	62
10.5. Hạn chế của công cụ IDE truyền thống.....	62

10.5.1. Khả năng của các IDE phổ biến (VS Code, PyCharm, v.v.)	62
10.5.2. Những hạn chế của công cụ truyền thống	62
10.5.3. Sự trỗi dậy của GenAI trong Refactoring	63
10.6. Ứng dụng GenAI trong Refactoring	63
10.6.1. Kỹ thuật Chuỗi tư duy (Chain-of-Thought - CoT)	63
10.6.2. Refactoring tương tác với ChatGPT và Copilot	63
10.6.3. Tự động hóa quy mô lớn với OpenAI API	64
10.7. Refactoring hiệu năng với GenAI	64
10.7.1. Chuyển đổi tư duy: Từ Scalar sang Vector	64
10.7.2. Lợi ích của việc Refactor hiệu năng qua GenAI	65
10.7.3. Vai trò của GenAI trong quy trình này	65
10.8. So sánh các công cụ GenAI	65
10.9. Kết luận	66
10.9.1. Sự chuyển dịch về công cụ hỗ trợ	66
10.9.2. Triết lý phát triển bền vững	66
10.9.3. Thông điệp cuối cùng	66
CHƯƠNG 11: FINE-TUNING MÔ HÌNH VỚI OPENAI	67
11.1. Giới thiệu	67
11.2. Tổng quan về Fine-tuning	67
11.2.1. Khái niệm và Bản chất Kỹ thuật	67
11.2.2. Mục tiêu chiến lược của Fine-tuning	67
11.2.3. So sánh nhanh: Prompt Engineering vs. Fine-tuning	68
11.3. So sánh Fine-tuning và Few-shot Learning	68
11.4. Quy trình Fine-tuning	69
11.5. Thiết kế dữ liệu huấn luyện	70
11.5.1. Cấu trúc một bản ghi mẫu	70
11.5.2. Ý nghĩa và vai trò của các thành phần	70
11.6. Contrastive Learning trong Fine-tuning	71
11.6.1. Khái niệm và Bản chất	71
11.6.2. Ứng dụng cụ thể qua Trọng số (Weight)	71
11.6.3. Lợi ích vượt trội	71

11.7. Case Study: Sinh code sạch (Clean Code Generation)	72
11.7.1. Những vấn đề tồn đọng của Mô hình mặc định (Base Model).....	72
11.7.2. Giải pháp thông qua Fine-tuning chuyên biệt	72
11.7.3. Kết quả thực nghiệm	72
11.8. Chi phí Fine-tuning	73
11.8.1. Chi phí Huấn luyện (Training Costs)	73
11.8.2. Chi phí Vận hành (Inference Costs)	73
11.8.3. Bảng so sánh tổng hợp	74
11.9. Đánh giá hiệu quả.....	74
11.10. Khi nào nên sử dụng Fine-tuning.....	74
11.10.1. Các kịch bản nên sử dụng Fine-tuning	74
11.10.2. Các kịch bản không nên sử dụng Fine-tuning	75
11.11. Kết luận	75
11.11.1. Những giá trị cốt lõi mang lại.....	75
DANH MỤC TÀI LIỆU THAM KHẢO	77

DANH MỤC HÌNH ẢNH

Hình 1: Kiến trúc LLM.....	14
Hình 2: Code minh hoạt Khoảng cách Manhattan.....	23
Hình 3: Cơ chế vận hành với lệnh explain/.....	24
Hình 4: Prompt chi tiết cho hàm tính khoảng cách Manhattan.....	26
Hình 5: Kết quả hàm tính khoảng cách Manhattan được ChatGPT trả về	28
Hình 6: Prompt có ngữ cảnh	30
Hình 7: Chuẩn hóa Coding style.....	37
Hình 8: Mô hình học quy tắc coding style.....	37
Hình 9: Chuẩn hóa logging và comment	38
Hình 10: Ví dụ về Iterative Programming	39
Hình 11: Template -based Prompting	40
Hình 12: Ví dụ về Chain-of-Thought Prompting.....	40
Hình 13: Github Coipilot Baseline	42
Hình 14: ChatGPT Baseline.....	43
Hình 15: Output của ChatGPT Baseline	43
Hình 16: OpenAI API Baseline.....	44
Hình 17: Output của OpenAI API Baseline	45
Hình 18: Sử dụng CoT (Chain of Thought) với GitHub Copilot.....	46
Hình 19: Copilot đề xuất cài đặt cho các hàm thành phần.....	47
Hình 20: CoT với ChatGPT	47
Hình 21: Kết quả đầu ra của CoT với ChatGPT	48
Hình 22: CoT với OpenAI API	49
Hình 23: Kết quả đầu ra của CoT với OpenAI API	49

Hình 24: Chuẩn hóa docstring	51
Hình 25: Chuẩn hóa docstring (v2).....	52
Hình 26: Lưu lại lịch sử Prompt và phản hồi.....	53
Hình 27: Giải pháp Selective History	54
Hình 28: Giải pháp Selective History (v2)	54
Hình 29: Giải pháp Selective History (v3)	54
Hình 30: Áp dụng Prompt Chaining với GitHub Copilot.....	56
Hình 31: Định hướng Refactoring	61
Hình 32: Quy trình Fine-Tuning với OpenAI	69

DANH MỤC BẢNG

Bảng 1: Phân tích ưu/nhược điểm của việc tích hợp Github Copilot vào quy trình phát triển.....	25
Bảng 2: Bảng so sánh các công cụ AI.....	34
Bảng 3: Bảng so sánh các công cụ GenAI.....	65
Bảng 4: Bảng so sánh nhanh giữa Prompt Engineering và Fine-Tuning.....	68
Bảng 5: Bảng so sánh Fine-Tuning và Few-shot Learning.....	68
Bảng 6: Bảng đánh giá kết quả thực nghiệm trước và sau quá trình Fine-Tuning	73
Bảng 7: Bảng so sánh Model gốc (Prompt Engineering) và Model Fine-Tuned.....	74
Bảng 8: Bảng đánh giá hiệu quả giữa model gốc và model sau Fine-Tuned.....	74

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	MSSV	Nội dung công việc
1	Nguyễn Hữu Anh Khoa	3122411098	- Nhóm trưởng - Phân chia công việc, chuẩn bị cấu trúc mã nguồn, nội dung mẫu - Thực hiện Lab và soạn nội dung và thực hiện Chương 10,11
2	Võ Thành Danh	3122411024	- Thực hiện Lab - Soạn nội dung Chương 8,9
3	Trang Gia Huy	3122411068	- Thực hiện Lab - Soạn nội dung Chương 6 và báo cáo tổng
4	Nguyễn Lê Quỳnh Hương	3122411078	- Thực hiện Lab - Soạn nội dung Chương 7 và slide thuyết trình

LỜI MỞ ĐẦU

Trong những năm gần đây, cuộc cách mạng về Trí tuệ nhân tạo (AI), đặc biệt là Trí tuệ nhân tạo tạo sinh (Generative AI - GenAI), đã tạo nên những thay đổi mang tính bước ngoặt trong nhiều lĩnh vực của đời sống xã hội. Đối với lĩnh vực kỹ thuật phần mềm, GenAI không chỉ dừng lại ở vai trò là một công cụ hỗ trợ mà đã trở thành một người bạn đồng hành, làm thay đổi tư duy và quy trình làm việc truyền thống của các lập trình viên.

Việc ứng dụng các mô hình ngôn ngữ lớn (LLMs) như GPT của OpenAI hay các công cụ hỗ trợ mã nguồn như GitHub Copilot đã mở ra những cơ hội chưa từng có. Tuy nhiên, để làm chủ được những công nghệ này một cách hiệu quả, đảm bảo chất lượng mã nguồn và tối ưu hóa chi phí vận hành, người kỹ sư cần có một cách tiếp cận hệ thống, từ việc hiểu rõ cơ chế hoạt động của API đến việc rèn luyện kỹ năng đặt câu lệnh (Prompt Engineering).

Báo cáo này được thực hiện nhằm mục đích nghiên cứu và thực hành quá trình ứng dụng GenAI vào toàn bộ chu kỳ phát triển phần mềm (SDLC). Nội dung báo cáo sẽ đi sâu vào các khía cạnh:

- Tổng quan về cơ hội của GenAI trong việc chuyển đổi từ tự động hóa đơn thuần sang hỗ trợ toàn diện vòng đời phần mềm.
- Kỹ thuật tương tác với OpenAI API thông qua lập trình Python, quản lý tài nguyên và chi phí.
- Thực hành sử dụng GitHub Copilot trên các môi trường phát triển phổ biến (VS Code, PyCharm, Jupyter Notebook).
- Xây dựng bộ quy tắc (Best Practices) để tối ưu hóa hiệu quả khi đặt câu hỏi cho ChatGPT và các công cụ hỗ trợ.

Thông qua quá trình nghiên cứu và thực hiện các bài thực hành (Labs) dựa trên tài liệu “Supercharged Coding with GenAI” của NXB Packt, báo cáo hy vọng sẽ đưa ra cái nhìn thực tiễn và những kỹ năng cần thiết để nâng cao năng suất lập trình trong kỷ nguyên AI.

Do kiến thức về lĩnh vực này vẫn đang phát triển không ngừng, báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ Thầy để nội dung được hoàn thiện hơn.

CHƯƠNG 6: CƠ CHẾ HOẠT ĐỘNG CỦA LLM

6.1. Giới thiệu

Trong những năm vừa qua, các công cụ trí tuệ nhân tạo như Chat GPT hay GitHub Copilot đã thay đổi hoàn toàn cách chúng ta làm việc và sáng tạo. Tuy nhiên, để thực sự làm chủ và khai thác tốt những công cụ này, chúng ta cần hiểu được "cỗ máy" bên trong chúng vận hành như thế nào.

Báo cáo này sẽ giúp bạn khám phá thế giới của các mô hình ngôn ngữ lớn theo một cách gần gũi nhất. Trước hết, chúng ta sẽ tìm hiểu về nguyên lý cơ bản để một cỗ máy có thể học và hiểu được ngôn ngữ của con người. Sau đó, bài báo cáo sẽ mô phỏng cấu trúc tổng thể giống như bộ khung xương giúp AI liên kết các thông tin lại với nhau.

6.2. Nguyên lý hoạt động của LLM

6.2.1. Bản chất của LLM

Thay vì hiểu ý nghĩa sâu xa của các khái niệm, LLM hoạt động hoàn toàn dựa trên xác suất thống kê. Qua quá trình được đọc hàng tỷ trang văn bản, chúng học được quy luật rằng: "Sau từ A và từ B, thì khả năng cao từ C sẽ xuất hiện".

Nhiệm vụ cốt lõi của chúng chỉ đơn giản là dự đoán phần tử tiếp theo (thường gọi là các "token" – có thể là một từ hoặc một phần của từ).

Để tạo ra một câu trả lời hoàn chỉnh, mô hình thực hiện một vòng lặp liên tục qua các bước sau:

Tiếp nhận yêu cầu (Prompt): Đầu tiên, máy tính sẽ nhận câu lệnh hoặc câu hỏi từ bạn. Đây là dữ liệu đầu vào để nó bắt đầu "suy luận".

Phân tích ngữ cảnh: Thay vì chỉ nhìn vào từ cuối cùng, AI sẽ nhìn lại toàn bộ các từ bạn đã viết để hiểu bạn đang muốn nói về chủ đề gì (đang hỏi về nấu ăn, lập trình hay tâm sự).

Dự đoán quân cờ tiếp theo: Dựa trên kho kiến thức khổng lồ đã học, nó sẽ tính toán xem từ nào có xác suất xuất hiện cao nhất để nối tiếp vào câu đang viết.

Vòng lặp không ngừng: Sau khi chọn được một từ, nó lại thêm từ đó vào văn bản và tiếp tục dự đoán từ kế tiếp nữa. Quá trình này lặp đi lặp lại liên tục cho đến khi tạo ra một dấu chấm hết câu hoặc đạt đến giới hạn cho phép.

6.2.2. Từ mô hình thống kê đến AI

Bài toán về sự bùng nổ của ngôn ngữ

Về mặt lý thuyết, ta có thể mô hình hóa ngôn ngữ bằng một công thức toán học đơn giản: $P(\text{từ tiếp theo} \mid \text{ngữ cảnh})$. Hiểu đơn giản là: "Xác suất để từ X xuất hiện sau một chuỗi từ cho trước là bao nhiêu?".

Tuy nhiên, thực tế lại nảy sinh một vấn đề khổng lồ:

Nếu bạn chỉ dự đoán dựa trên 1-2 từ phía trước, kết quả sẽ rất ngô nghê.

Để nói chuyện thông minh, AI cần nhìn lại hàng trăm, hàng ngàn từ trước đó.

Với số lượng từ vựng khổng lồ của con người, việc ghi nhớ tất cả các tổ hợp từ có thể xảy ra để tính xác suất là điều không tưởng. Không có máy tính nào đủ bộ nhớ để lưu trữ một "cuốn từ điển" khổng lồ như vậy.

Giải pháp: Mạng thần kinh (Neural Network)

Thay vì cố gắng ghi nhớ từng tổ hợp từ cụ thể, các nhà khoa học đã sử dụng Mạng thần kinh nhân tạo để "ước lượng" hoặc "xấp xỉ" hàm xác suất đó.

Thay vì lưu trữ một bảng dữ liệu cứng nhắc, Mạng thần kinh hoạt động như một bộ lọc thông minh:

Học quy luật, không học vẹt: Nó không nhớ máy móc cả câu văn, mà học cách các từ tương tác với nhau thông qua các con số (gọi là trọng số).

Nén thông tin: Nó nén toàn bộ tri thức từ hàng tỷ văn bản vào một mô hình gọn nhẹ hơn. Khi bạn đưa vào một ngữ cảnh mới, mạng thần kinh sẽ tự tính toán và đưa ra dự đoán từ tiếp theo một cách linh hoạt, ngay cả khi nó chưa từng thấy chính xác câu văn đó trước đây.

Nói cách khác, Mạng thần kinh giúp AI chuyển từ việc "tra bảng xác suất" sang việc "cảm nhận quy luật" của ngôn ngữ.

6.3. Kiến trúc tổng thể của LLM

6.3.1. Mô tả tổng quan

Ta có thể hình dung toàn bộ quá trình một LLM tạo ra câu chữ giống như một dây chuyền sản xuất hiện đại (pipeline). Thay vì "đọc và hiểu" theo cách của con người, hệ thống này thực hiện một chuỗi các bước biến đổi toán học rất logic để biến ngôn ngữ thành những con số mà máy tính có thể tính toán được.

1. Chia nhỏ dữ liệu (Tokenization)

Bước đầu tiên, máy tính không nhìn cả một câu dài mà sẽ chia nhỏ câu đó thành những mẫu thông tin bé hơn gọi là token. Một token có thể là một từ nguyên vẹn, một phần của từ, hoặc thậm chí chỉ là một dấu phẩy. Việc này giúp máy tính quản lý vốn từ vựng khổng lồ một cách hiệu quả hơn.

2. Biến chữ thành số (Embedding)

Máy tính không thể làm toán trên chữ cái, vì vậy mỗi token sẽ được gán cho một dãy số định danh, gọi là vector.

Hãy tưởng tượng mỗi từ giống như một tọa độ trong một không gian đa chiều.

Những từ có ý nghĩa gần nhau (như "vua" và "hoàng hậu") sẽ được đặt nằm gần nhau trong không gian số này.

3. Học mối quan hệ qua các lớp Transformer

Đây là "trái tim" của hệ thống. Dữ liệu số này sẽ đi qua rất nhiều tầng lọc phức tạp (gọi là các lớp Transformer). Ở đây, AI không chỉ nhìn từng từ riêng lẻ mà nó xem xét mối quan hệ giữa các từ trong câu.

Ví dụ: Trong câu "Con chuột này bị hỏng phím", nhờ các lớp này mà AI hiểu "con chuột" ở đây là thiết bị máy tính chứ không phải con vật.

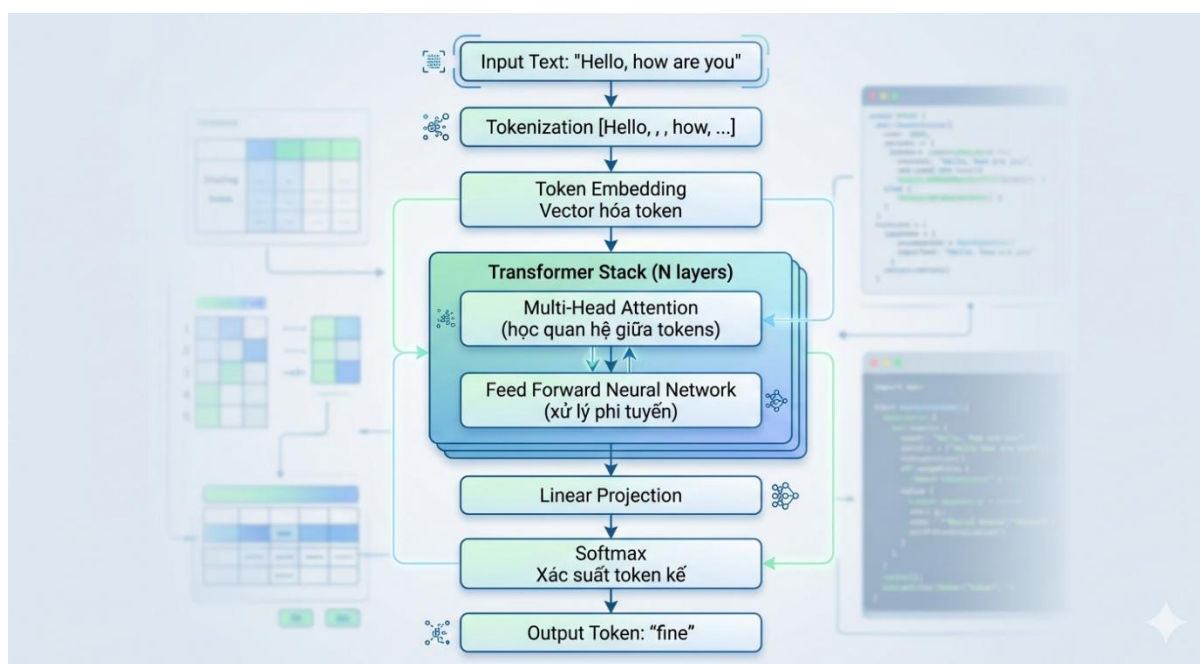
4. Dự đoán và Hoàn thiện

Cuối cùng, sau khi đã "nhào nặn" các con số qua nhiều tầng, lớp cuối cùng của mô hình sẽ đưa ra một bảng dự đoán: "Với ngữ cảnh này, từ tiếp theo có xác suất cao nhất là gì?".

Nó chọn ra từ đó, đặt vào câu, rồi lại đưa toàn bộ câu mới này quay lại đầu dây chuyền để dự đoán từ tiếp theo nữa.

Quá trình này lặp đi lặp lại cho đến khi tạo ra một văn bản hoàn chỉnh và tự nhiên nhất.

6.3.2 Diagram kiến trúc LLM (chi tiết)



Hình 1: Kiến trúc LLM

Diagram trên cho thấy rõ một điểm quan trọng: LLM không phải là một khối duy nhất, mà là một chuỗi các thành phần phối hợp chặt chẽ.

Tokenization đóng vai trò chuyển đổi dữ liệu từ dạng ngôn ngữ sang dạng mà máy có thể xử lý.

Embedding biến mỗi token thành một vector trong không gian nhiều chiều, nơi mà khoảng cách giữa các vector phản ánh mối quan hệ ngữ nghĩa.

Transformer Stack là phần “bộ não” chính, nơi mô hình học được cách các từ liên kết với nhau trong câu.

Cuối cùng, Softmax biến kết quả thành phân phối xác suất để chọn ra token tiếp theo.

Điều đáng chú ý là toàn bộ hệ thống này hoạt động dựa trên toán học và tối ưu hóa, không có bất kỳ “kiến thức tường minh” nào như trong cơ sở dữ liệu truyền thống.

6.4. Tokenization (Phân tích chuyên sâu)

6.4.1 Token là gì?

Trong kiến trúc của các mô hình ngôn ngữ lớn (LLM), Token được định nghĩa là đơn vị dữ liệu nhỏ nhất mà hệ thống có thể tiếp nhận và xử lý. Thay vì đọc hiểu nguyên văn các từ ngữ phức tạp như con người, mô hình sẽ thực hiện quy trình "băm nhỏ" văn bản đầu vào thành một chuỗi các mã hiệu riêng biệt.

Cơ chế chia cắt văn bản

Cách thức một từ được phân rã thành token phụ thuộc vào độ phổ biến và cấu trúc của từ đó trong bộ dữ liệu huấn luyện:

Đối với các thuật ngữ thông dụng: Mô hình có thể nhận diện toàn bộ từ đó là một token duy nhất để tối ưu hóa tốc độ xử lý.

Ví dụ: Từ "hello" thường được giữ nguyên thành một đơn vị ["hello"].

Đối với các từ phức tạp, từ ghép hoặc tên riêng: Hệ thống sẽ chia nhỏ chúng thành các phần (sub-words). Việc này giúp AI có thể hiểu được ý nghĩa của các từ mới bằng cách kết hợp các mảnh ghép đã biết, đồng thời giúp bộ từ điển của máy tính gọn nhẹ hơn.

Ví dụ: Từ "ChatGPT" có thể được tách thành ["Chat", "G", "PT"].

Ví dụ: Từ "hello" trong một số ngữ cảnh kỹ thuật cũng có thể được chia thành ["he", "llo"].

6.4.2. Các phương pháp Tokenization

Để biến văn bản thành dữ liệu mà máy tính có thể xử lý, các nhà phát triển thường sử dụng ba cấp độ phân tách chính. Mỗi phương pháp đều có những ưu và nhược điểm riêng nhằm tối ưu hóa hiệu suất của mô hình:

1. Phân tách theo cấp độ ký tự (Character-level)

Ở phương pháp này, mỗi chữ cái hoặc ký tự đơn lẻ được tính là một token.

Đặc điểm: Bộ từ điển mà máy tính cần nhớ rất nhỏ (chỉ gồm các chữ cái trong bảng chữ cái).

Nhược điểm: Một câu văn ngắn khi chia ra sẽ tạo thành một chuỗi token quá dài, khiến máy tính tốn rất nhiều tài nguyên và thời gian để xử lý hết một đoạn văn bản.

2. Phân tách theo cấp độ từ (Word-level)

Mỗi từ nguyên vẹn sẽ được coi là một token duy nhất.

Đặc điểm: Chuỗi dữ liệu đầu vào ngắn gọn, giúp máy tính xử lý nhanh hơn.

Nhược điểm: Tạo ra một bộ từ vựng khổng lồ. Vì ngôn ngữ luôn biến đổi và có vô số từ mới (hoặc các biến thể của từ), máy tính sẽ không thể ghi nhớ hết tất cả các từ tồn tại trên thế giới.

3. Phân tách theo cấp độ mảnh từ (Subword)

Đây là phương pháp phổ biến nhất hiện nay (được sử dụng trong các kỹ thuật như BPE hay WordPiece). Thay vì chọn ký tự hay từ, hệ thống sẽ chia văn bản thành các mảnh nhỏ có nghĩa.

Ưu điểm vượt trội: Đây là giải pháp cân bằng hoàn hảo giữa độ dài chuỗi dữ liệu và kích thước bộ từ vựng.

Cách hoạt động: Những từ thông dụng sẽ được giữ nguyên, còn những từ hiếm hoặc từ mới sẽ được ghép lại từ các mảnh nhỏ đã biết. Điều này giúp AI vừa xử lý nhanh, vừa có thể "hiểu" được cả những từ mà nó chưa từng gặp trước đây.

6.4.3 Tại sao Tokenization quan trọng?

Việc lựa chọn độ dài và số lượng token đầu vào không chỉ là một thiết lập kỹ thuật đơn thuần mà còn là một quyết định chiến lược, ảnh hưởng trực tiếp đến ba yếu tố cốt lõi của một hệ thống trí tuệ nhân tạo:

1. Tốc độ xử lý (Hiệu suất)

Kích thước đầu vào càng lớn, khối lượng tính toán mà "mạng thần kinh" phải thực hiện càng tăng theo cấp số nhân.

Input ngắn: Giúp mô hình phản hồi gần như tức thì, phù hợp cho các ứng dụng trò chuyện trực tiếp.

Input dài: Khiến hệ thống mất nhiều thời gian hơn để "đọc" và phân tích toàn bộ ngữ cảnh trước khi đưa ra từ đầu tiên.

2. Chi phí vận hành

Trong thế giới AI, mỗi token đều có giá trị về tài chính.

Tài nguyên phần cứng: Việc xử lý các chuỗi token dài đòi hỏi bộ nhớ đồ họa (VRAM) và năng lượng tính toán cực lớn.

Chi phí sử dụng: Đối với các doanh nghiệp sử dụng dịch vụ thông qua API (như của OpenAI hay Google), số lượng token đầu vào càng lớn thì chi phí hóa đơn hàng tháng càng cao. Việc tối ưu hóa kích thước input chính là chìa khóa để tiết kiệm ngân sách.

3. Độ chính xác của kết quả

Đây là yếu tố quan trọng nhất về mặt chất lượng.

Ngữ cảnh đầy đủ: Nếu kích thước input quá nhỏ, AI sẽ bị "thiếu thông tin" và dễ đưa ra câu trả lời sai lệch hoặc lạc đề.

Giới hạn ghi nhớ: Tuy nhiên, nếu bắt AI xử lý một lượng token quá khổng lồ cùng lúc, nó có thể gặp hiện tượng "mất tập trung" vào các chi tiết ở giữa đoạn văn, dẫn đến việc giảm độ chính xác của câu trả lời cuối cùng.

6.4.4. Embedding

Sau khi văn bản đã được chia nhỏ thành các Token, bước tiếp theo là chuyển những mẫu thông tin này thành một định dạng mà máy tính có thể tính toán được. Quá trình này được gọi là Embedding (nhúng ngôn ngữ).

1. Biến chữ viết thành dãy số

Trong giai đoạn này, mỗi token (ví dụ như từ "cat") sẽ không còn là các ký tự chữ cái đơn thuần. Thay vào đó, nó được đại diện bởi một dãy số thực dài (được gọi là vector).

Ví dụ: Từ "cat" có thể được mã hóa thành một chuỗi số như [0.2, -0.5, 0.8, ...].

2. Vai trò của Embedding trong việc hiểu ngôn ngữ

Việc chuyển đổi sang các dãy số này mang lại hai lợi ích cốt lõi cho mô hình:

So sánh ngữ nghĩa: Nhờ các con số, AI có thể thực hiện các phép tính toán học để đo lường khoảng cách giữa các từ. Những từ có ý nghĩa tương đồng (như "mèo" và "hổ") sẽ có các dãy số gần giống nhau, giúp AI hiểu rằng chúng thuộc cùng một nhóm chủ đề.

Hiểu mối quan hệ giữa các từ: Embedding giúp mô hình nắm bắt được các mối quan hệ logic phức tạp.

Ví dụ điển hình: AI có thể hiểu được mối liên hệ tương quan kiểu: "Vua" - "Nam giới" + "Nữ giới" = "Hoàng hậu".

Nói cách khác, Embedding chính là cách chúng ta xây dựng một "bản đồ ý nghĩa" khổng lồ, nơi mỗi từ ngữ đều có một vị trí xác định, giúp AI không chỉ thấy mặt chữ mà còn thấy được cả mối liên hệ ẩn sâu bên dưới chúng.

6.5. Attention (Phân tích chuyên sâu)

6.5.1 Vấn đề cần giải quyết

Trong quá trình xử lý ngôn ngữ, một trong những thách thức lớn nhất đối với máy tính là việc xác định mối liên kết giữa các từ ngữ nằm cách xa nhau trong cùng một câu hoặc một đoạn văn dài. Cơ chế Attention (Sự chú ý) chính là giải pháp đột phá giúp AI vượt qua rào cản này.

Khi chúng ta đọc một câu văn, bộ não con người tự động liên kết các đại từ hoặc tính từ với đối tượng mà chúng đang mô tả. Tuy nhiên, đối với máy tính, việc đọc từng từ theo thứ tự từ trái sang phải thường khiến nó "quên" mất các thông tin quan trọng đã xuất hiện ở phía trước.

Hãy xem xét ví dụ sau:

"The cat sat on the mat because it was tired."

Trong câu này, từ "it" (nó) xuất hiện ở cuối câu. Về mặt cấu trúc, từ này nằm ngay sát từ "mat" (chiếc thảm), nhưng về mặt ý nghĩa, nó lại dùng để chỉ "cat" (con mèo).

Nếu không có cơ chế đặc biệt, AI có thể hiểu lầm rằng "chiếc thảm" đang bị mệt.

Thử thách đặt ra là: Làm sao để AI biết "tập trung" vào từ "cat" khi đang xử lý từ "it", dù giữa chúng có rất nhiều từ ngăn cách?

6.5.2 Cơ chế Attention

Sau khi đã xác định được các từ cần liên kết, mô hình sẽ thực hiện một quá trình tính toán để đưa ra quyết định: Nên ưu tiên tập trung vào đâu?

Vai trò định lượng mức độ quan trọng

Cơ chế Attention cho phép mô hình gán một "trọng số" cho từng token trong câu.

- Thay vì xem mọi từ đều có giá trị như nhau, AI sẽ tự hỏi: "Để hiểu từ hiện tại, tôi cần bao nhiêu phần trăm thông tin từ các từ đứng trước và đứng sau nó?".
- Kết quả là một bảng phân bổ sự chú ý, giúp mô hình lọc bỏ những từ gây nhiễu và giữ lại những từ mang tính quyết định đến ý nghĩa của toàn câu.

Công thức toán học cốt lõi

Để thực hiện việc này một cách chính xác, các nhà khoa học sử dụng một công thức đặc trưng để tính toán mối quan hệ giữa ba thành phần chính (được ví như một hệ thống tìm kiếm thông tin):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó, bạn có thể hiểu đơn giản các ký hiệu này như sau:

- **Q (Query - Câu hỏi):** Đại diện cho từ mà mô hình đang xem xét.
- **K (Key - Từ khóa):** Đại diện cho tất cả các từ khác trong câu để đối chiếu.
- **V (Value - Giá trị):** Chứa đựng thông tin thực sự của các từ đó.

Quy trình tính toán diễn ra như sau:

1. Mô hình lấy "Câu hỏi" (Q) nhân với các "Từ khóa" (K) để xem chúng khớp nhau đến mức nào.
2. Kết quả được đưa qua hàm Softmax để chuyển thành các tỷ lệ phần trăm (xác suất). Tổng các tỷ lệ này luôn bằng 100%.
3. Cuối cùng, tỷ lệ này được nhân với "Giá trị" (V) để trích xuất ra thông tin quan trọng nhất.

Nói một cách ví von, quá trình này giống như bạn đang đứng trong một thư viện: bạn cầm một tờ giấy ghi chủ đề mình cần (Query), đi lướt qua các nhãn sách trên kệ (Key) để tìm cuốn sách phù hợp nhất, và cuối cùng mở cuốn sách đó ra để lấy kiến thức (Value).

6.5.3 Multi-head Attention

Thay vì chỉ sử dụng một bộ lọc "Attention" duy nhất, các mô hình ngôn ngữ hiện đại triển khai nhiều bộ lọc hoạt động song song. Kỹ thuật này được ví như việc quan sát một bức tranh thông qua nhiều lăng kính khác nhau để không bỏ lỡ bất kỳ chi tiết nào.

Tại sao cần nhiều "đầu" chú ý?

Trong một câu văn, mối quan hệ giữa các từ rất đa tầng và phức tạp. Việc sử dụng nhiều "Head" (đầu chú ý) cho phép mô hình học được các khía cạnh khác nhau của ngôn ngữ đồng thời:

- **Học về Ngữ pháp:** Một số đầu chú ý sẽ tập trung vào việc kiểm tra cấu trúc câu, như việc nối động từ với chủ ngữ hoặc xác định các dấu câu quan trọng.
- **Học về Ngữ nghĩa:** Các đầu khác lại đi sâu vào việc hiểu ý nghĩa thực sự của từ. Ví dụ, nó sẽ liên kết từ "ngân hàng" với "tài chính" thay vì "bờ sông" dựa trên các từ xung quanh.
- **Học về Vị trí:** Một số đầu chuyên trách việc ghi nhớ thứ tự các từ, giúp AI phân biệt được sự khác biệt giữa "Chó đuổi mèo" và "Mèo đuổi chó".

Hiệu quả mang lại

Bằng cách tổng hợp thông tin từ tất cả các đầu chú ý này, mô hình có được một cái nhìn toàn diện và sâu sắc hơn về văn bản. Kết quả cuối cùng không chỉ là sự kết hợp máy móc mà là một sự hiểu biết đa chiều, giúp AI xử lý được những câu nói lắt léo, mỉa mai hoặc các văn bản chuyên ngành phức tạp.

Nói cách khác, nếu một đầu chú ý giống như một người quan sát, thì Multi-Head Attention giống như một hội đồng chuyên gia, nơi mỗi người phụ trách một chuyên môn riêng để cùng đưa ra một kết luận chính xác nhất.

6.6. Hạn chế của LLM

Mặc dù có khả năng tạo ra văn bản ấn tượng, các mô hình LLM vẫn tồn tại những rào cản kỹ thuật và bản chất mà người dùng cần lưu ý để tránh kỳ vọng sai lệch.

6.6.1. Thiếu sự hiểu biết thực sự về thế giới

Như đã phân tích, LLM vận hành dựa trên xác suất thống kê chứ không có tư duy hay ý thức.

- AI không thực sự "biết" một khái niệm là gì; nó chỉ biết rằng các từ ngữ đó thường xuất hiện cùng nhau.
- Điều này dẫn đến việc AI có thể viết rất trôi chảy nhưng lại hoàn toàn thiếu đi sự logic thực tế hoặc trải nghiệm cảm xúc của con người.

6.6.2. Hiện tượng "Ảo giác" (Hallucination)

Đây là một trong những thách thức lớn nhất hiện nay. Do mục tiêu chính của mô hình là dự đoán "từ tiếp theo có vẻ hợp lý nhất", đôi khi nó sẽ tự tin đưa ra những thông tin sai sự thật nhưng được trình bày một cách rất thuyết phục. AI có thể tự bịa ra các sự kiện lịch sử, tên sách hoặc các công thức khoa học không hề tồn tại.

6.6.3. Sự phụ thuộc vào dữ liệu huấn luyện

Chất lượng đầu ra của AI tỉ lệ thuận với chất lượng dữ liệu mà nó được học.

- Định kiến: Nếu dữ liệu huấn luyện chứa các quan điểm lệch lạc hoặc phân biệt đối xử, AI sẽ lặp lại những định kiến đó.
- Thời điểm cập nhật: AI chỉ biết những gì đã xảy ra cho đến thời điểm dữ liệu được thu thập. Nó hoàn toàn "mù tịt" về các sự kiện diễn ra sau khi quá trình huấn luyện kết thúc (Knowledge Cutoff).

6.6.4. Giới hạn về "Cửa sổ ngữ cảnh" (Context Window)

Mọi mô hình LLM đều có một giới hạn về lượng thông tin mà nó có thể "ghi nhớ" trong một cuộc trò chuyện.

- Khi bạn đưa vào một tài liệu quá dài vượt quá giới hạn này, AI sẽ bắt đầu "quên" những phần nội dung ở đầu văn bản.
- Việc xử lý các ngữ cảnh cực lớn không chỉ đòi hỏi tài nguyên máy tính khổng lồ mà còn làm giảm độ chính xác khi truy xuất chi tiết ở giữa đoạn văn.

6.7. Kết luận

Nhìn một cách tổng thể, các mô hình ngôn ngữ lớn (LLM) không phải là một "phép màu" mà là thành quả của sự kết hợp chặt chẽ giữa ba trụ cột khoa học: Toán học xác suất, Học máy (Machine Learning) và Học sâu (Deep Learning) với kiến trúc Transformer đóng vai trò trung tâm.

Trình tự vận hành cốt lõi

Mọi câu trả lời thông minh mà chúng ta nhận được đều là kết quả của một dây chuyền xử lý nghiêm ngặt:

1. Tokenization: Chia nhỏ ngôn ngữ thành các mảnh ghép.
2. Embedding: Biến các mảnh ghép thành tọa độ số mang ý nghĩa.
3. Attention: Kết nối các từ ngữ để hiểu đúng ngữ cảnh.
4. Prediction: Dự đoán từ tiếp theo dựa trên xác suất cao nhất.

Tại sao việc hiểu rõ LLM lại quan trọng?

Việc nắm vững các thành phần kỹ thuật này không chỉ dành cho các chuyên gia, mà còn mang lại lợi ích thực tế cho mọi người dùng:

1. Sử dụng AI hiệu quả hơn: Khi hiểu cách AI "tư duy" qua token và ngữ cảnh, bạn sẽ biết cách viết câu lệnh (prompt) rõ ràng, giúp máy tính đưa ra câu trả lời chính xác nhất.
2. Thiết kế và nâng cấp hệ thống: Đây là nền tảng bắt buộc nếu bạn muốn xây dựng các ứng dụng nâng cao như RAG (truy xuất dữ liệu riêng tư cho AI) hoặc Fine-tune (huấn luyện bổ sung) để biến AI thành một chuyên gia trong lĩnh vực cụ thể.
3. Tránh hiểu sai khả năng của AI: Hiểu về xác suất và giới hạn dữ liệu giúp chúng ta có cái nhìn thực tế, không quá kỳ vọng vào sự "thông minh" của máy tính và luôn có sự kiểm chứng để tránh những sai lầm từ hiện tượng "ảo giác" của AI.

CHƯƠNG 7: ĐỌC VÀ HIỂU CODEBASES VỚI GENAI

7.1. Giới thiệu

Trong phát triển phần mềm hiện đại, việc đọc và hiểu một codebase lớn là kỹ năng quan trọng, đặc biệt khi làm việc trong các dự án thực tế. Không giống như các ví dụ đơn lẻ, codebase thường bao gồm nhiều thành phần như file nguồn, cấu hình môi trường, và tài liệu.

Chương này tập trung vào việc sử dụng các công cụ GenAI (GitHub Copilot, ChatGPT, OpenAI API) để hỗ trợ quá trình đọc, phân tích và hiểu code một cách hiệu quả hơn.

7.2. Cấu trúc Codebase

Một codebase điển hình bao gồm:

- `app.py`: Điểm bắt đầu của ứng dụng
- `src/`: Chứa business logic
- `requirements.txt`: Danh sách thư viện
- `Dockerfile`: Cấu hình môi trường chạy
- `README.md`: Tài liệu hướng dẫn

Ví dụ cấu trúc:

ch7/

```
|— app.py
|— src/
|   |— manhattan.py
|— Dockerfile
|— README.md
|— requirements.txt
```

Việc hiểu cấu trúc này giúp xác định:

Việc tuân thủ cấu trúc này mang lại ba lợi ích chiến lược cho công tác quản trị dự án:

- Xác định rõ Entry point: Giúp lập trình viên nhanh chóng nắm bắt được điểm bắt đầu của luồng thực thi.

- Minh bạch luồng dữ liệu: Sự phân tách giữa app.py và src/ giúp theo dõi cách dữ liệu được tiếp nhận, xử lý và phản hồi một cách trực quan.
- Quản lý Dependencies chặt chẽ: Đảm bảo hệ thống luôn có đầy đủ các thành phần hỗ trợ cần thiết, giảm thiểu rủi ro xung đột phần mềm khi triển khai thực tế.

7.3. Business Logic: Manhattan Distance

7.3.1 Khái niệm

Khoảng cách Manhattan (hay còn gọi là Taxicab geometry) là một phương pháp đo lường khoảng cách giữa hai điểm trong không gian n-chiều. Thay vì đo đường chéo trực tiếp như khoảng cách Euclidean, Manhattan Distance tính tổng độ lệch tuyệt đối giữa các tọa độ tương ứng của chúng.

Công thức toán học được xác định như sau:

$$d(x,y)=\sum_{i=1}^n|x_i-y_i|$$

Trong đó:

- $d(x, y)$: Khoảng cách Manhattan giữa hai điểm.
- $\sum_{i=1}^n$: Ký hiệu tổng từ chiều thứ 1 đến chiều thứ n.
- $|x_i - y_i|$: Giá trị tuyệt đối của hiệu giữa tọa độ thứ i của điểm x và điểm y.

Trong thực tế, phép đo này đóng vai trò quan trọng trong nhiều lĩnh vực:

- Machine Learning: Được sử dụng làm hàm đo độ tương đồng (similarity measure) trong các thuật toán phân cụm hoặc phân loại, đặc biệt hiệu quả với dữ liệu có nhiều biến rời rạc.
- Xử lý hình ảnh (Image Processing): Hỗ trợ trong việc nhận dạng mẫu và so sánh sự khác biệt giữa các pixel hoặc đặc trưng ảnh.

7.3.2 Code minh họa

```
def get_manhattan_distance(
    x1: Tuple[int, int],
    x2: Tuple[int, int],
) -> int:
    x_dist: int = abs(x1[0] - x2[0])
    y_dist: int = abs(x1[1] - x2[1])

    dist: int = x_dist + y_dist
    return dist
```

Hình 2: Code minh hoạt Khoảng cách Manhattan

- (df1 - df2): tính hiệu từng phần tử
- .abs(): lấy giá trị tuyệt đối
- .sum().sum(): cộng toàn bộ giá trị

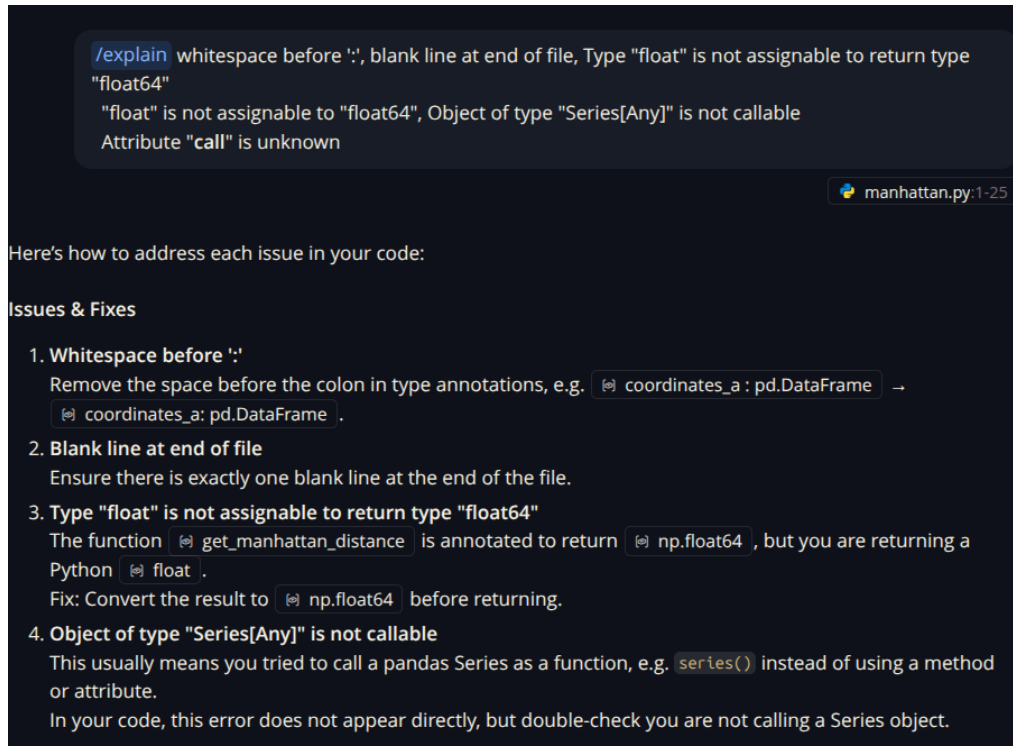
7.4. SỬ DỤNG GENAI ĐỂ HIỂU CODE

Trong quá trình xây dựng và bảo trì codebase, GitHub Copilot được sử dụng như một trợ lý AI đắc lực, giúp tối ưu hóa thời gian đọc hiểu và kiểm soát mã nguồn.

7.4.1. Cơ chế vận hành với lệnh /explain

Công cụ này cho phép lập trình viên tương tác trực tiếp với mã nguồn thông qua giao diện chat hoặc câu lệnh đặc biệt. Khi sử dụng lệnh /explain trên một đoạn mã cụ thể (ví dụ: hàm `get_manhattan_distance`), AI sẽ thực hiện:

- Giải mã chức năng: Phân tích mục đích cốt lõi của hàm trong ngữ cảnh dự án.
- Làm rõ tham số và đầu ra: Giải thích chi tiết ý nghĩa của các đối số truyền vào (arguments) và kiểu dữ liệu trả về (return type).
- Phân tích logic dòng lệnh: Chia nhỏ các phương thức liên hoàn như `.abs().sum().sum()` để người dùng hiểu rõ tác động của từng bước



Hình 3: Cơ chế vận hành với lệnh explain/

7.4.2. Đánh giá hiệu quả ứng dụng

Việc tích hợp GitHub Copilot vào quy trình phát triển mang lại những giá trị và hạn chế nhất định:

Đặc điểm	Chi tiết
Ưu điểm	Tốc độ: Giảm thiểu thời gian tra cứu tài liệu thủ công cho các hàm phức tạp. Tính tích hợp: Hoạt động trực tiếp trên IDE (VS Code, JetBrains), giúp duy trì luồng tập trung (Flow state).
Nhược điểm	Độ sâu tùy chỉnh: Phản hồi của AI đôi khi mang tính tổng quát, chưa thể đi sâu vào các quy ước nghiệp vụ đặc thù hoặc các ràng buộc logic quá riêng biệt của doanh nghiệp.

Bảng 1: Phân tích ưu/nhược điểm của việc tích hợp Github Copilot vào quy trình phát triển

Kết luận

Sự kết hợp giữa một cấu trúc codebase chuẩn mực và các công cụ hỗ trợ thông minh như GitHub Copilot giúp đội ngũ kỹ thuật không chỉ đẩy nhanh tiến độ phát triển mà còn đảm bảo tính kế thừa cao cho mã nguồn.

7.4.3. ChatGPT

Để nhận được sự hỗ trợ chính xác nhất từ các công cụ AI như GitHub Copilot hay ChatGPT, quy trình phát triển cần áp dụng các Prompt chi tiết. Việc cung cấp đầy đủ các thành phần thông tin giúp AI đưa ra mã nguồn hoặc lời giải thích có độ tin cậy cao.

7.4.3.1. Cấu trúc một Prompt chuẩn cho Logic Nghiệp vụ

Một prompt hiệu quả cho hàm tính khoảng cách Manhattan cần bao gồm 5 thành phần cốt lõi:

1. Purpose (Mục đích): Mô tả rõ hàm dùng để làm gì (ví dụ: Tính tổng khoảng cách tuyệt đối giữa hai tập dữ liệu).
2. Arguments (Đối số): Xác định kiểu dữ liệu đầu vào (Pandas DataFrame, Numpy Array, v.v.).
3. Data Flow (Luồng dữ liệu): Mô tả các bước xử lý (Trừ phần tử -> Lấy trị tuyệt đối -> Tổng hợp).
4. Output (Đầu ra): Định dạng kết quả trả về (Số thực float, Integer).
5. Edge Cases (Trường hợp biên): Các tình huống lỗi cần xử lý.

7.4.3.2. Ví dụ Prompt chi tiết cho hàm Manhattan Distance

Dưới đây là mẫu Prompt có thể sử dụng để yêu cầu AI viết hoặc tối ưu lại hàm trong dự án:

```
You, 3 days ago | 1 author (You)
1  CONTEXT: you are provided with:
2  1. A Python function enclosed with {{{ FUNCTION }}}
3  2. Explanation points enclosed with {{ POINTS }}}. You, 3 days ago
4
5  TASK: Explain the function using the explanation points.
6
7  FUNCTION: {{{ def get_manhattan_distance(
8  |         df1: pd.DataFrame,
9  |         df2: pd.DataFrame,
10 |     ) -> float:
11 |         element_wise_dist: pd.DataFrame = (df1 - df2).abs()
12 |         dist: Union[np.int64, np.float64] = element_wise_dist.sum().sum()
13 |         return float(dist)
14 |     }}}
15
16  POINTS: {{{
17  | 1. Function's purpose
18  | 2. Arguments and their types
19  | 3. Step-by-step data flow
20  | 4. Output and its types
21  | 5. Potential edge cases
22  | }}}
23
24  EXPLANATION:
```

Hình 4: Prompt chi tiết cho hàm tính khoảng cách Manhattan

1. Mục đích của Hàm (Purpose)

Hàm `get_manhattan_distance` được thiết kế để xác định độ tương đồng giữa hai tập dữ liệu bằng cách tính tổng các khoảng cách tuyệt đối giữa từng cặp phần tử tương ứng trong không gian đa chiều.

2. Đối số và Kiểu dữ liệu (Arguments & Types)

Hàm tiếp nhận hai đối số đầu vào với kiểu dữ liệu được định nghĩa chặt chẽ:

- `df1: pd.DataFrame`: Bảng dữ liệu thứ nhất.
- `df2: pd.DataFrame`: Bảng dữ liệu thứ hai (phải có cùng cấu trúc với `df1`).

3. Luồng dữ liệu chi tiết (Step-by-step Data Flow)

Quá trình xử lý dữ liệu bên trong hàm diễn ra qua ba giai đoạn chính:

Phép trừ phần tử: Thực hiện $(df1 - df2)$ để tìm độ lệch giữa hai ma trận dữ liệu.

- Lấy giá trị tuyệt đối: Sử dụng phương thức `.abs()` để chuyển đổi mọi độ lệch thành giá trị dương, tạo ra `element_wise_dist`.
- Tổng hợp hai cấp: Thực hiện `.sum().sum()` để cộng tổng theo cột, sau đó cộng các tổng cột lại thành một giá trị duy nhất (`dist`).

4. Kết quả đầu ra (Output & Types)

- Giá trị trả về: Một số thực đơn lẻ.
- Kiểu dữ liệu: float.

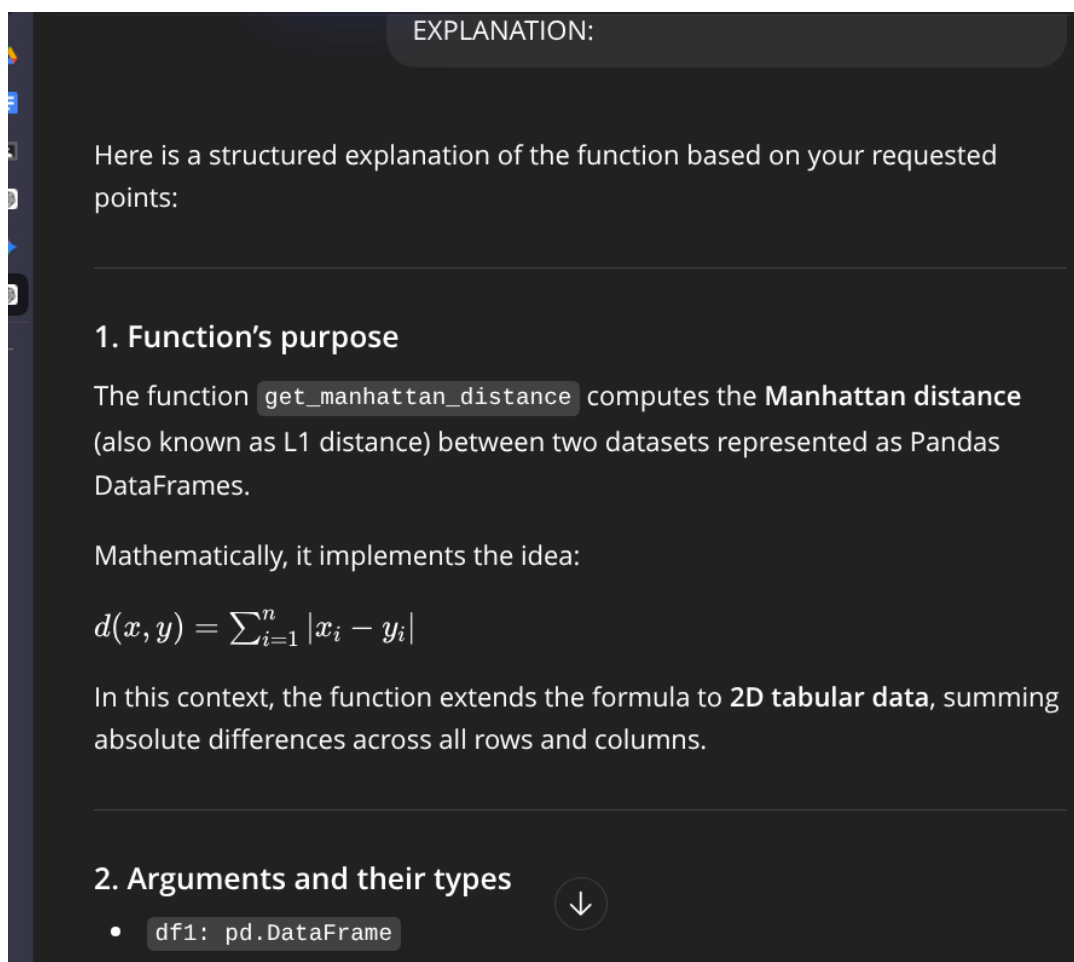
Lưu ý kỹ thuật: Biến tạm `dist` có thể thuộc kiểu `np.int64` hoặc `np.float64` tùy thuộc vào dữ liệu đầu vào, nhưng hàm luôn ép kiểu về float trước khi trả về để đảm bảo tính nhất quán cho các logic tiếp theo.

5. Các trường hợp biên cần lưu ý (Potential Edge Cases)

Mặc dù hàm đã hoạt động tốt trong điều kiện tiêu chuẩn, cần xem xét các kịch bản lỗi sau để tăng độ bền vững (Robustness):

- Sai lệch kích thước (Shape Mismatch): Nếu `df1` và `df2` không có cùng số hàng hoặc cột, phép trừ sẽ tạo ra các giá trị NaN không mong muốn.
- Dữ liệu thiếu (NaN values): Nếu một trong hai DataFrame chứa giá trị trống, kết quả cuối cùng có thể bị sai lệch hoặc trả về NaN.
- Dữ liệu không phải số: Nếu DataFrame chứa các cột dạng chuỗi (String), hàm sẽ gặp lỗi thực thi (TypeError) khi thực hiện phép trừ.

Kết quả được ChatGPT trả về:



Hình 5: Kết quả hàm tính khoảng cách Manhattan được ChatGPT trả về

7.4.3.3. Tầm quan trọng của việc xử lý Edge Cases

Việc xác định các trường hợp biên trong Prompt giúp mã nguồn trở nên bền vững (robust) hơn:

- Sai shape: Tránh các lỗi `ValueError` khi thực hiện phép tính ma trận không tương thích.
- NaN: Ngăn chặn việc kết quả trả về là NaN, gây sai lệch cho các bước phân tích sau.
- Data không phải số: Bảo vệ hàm khỏi các lỗi kiểu dữ liệu (`TypeError`) khi vô tình truyền vào dữ liệu dạng chuỗi (String).

7.4.3 OpenAI API

Trong các dự án có quy mô lớn với hàng trăm module, việc sử dụng các công cụ tương tác thủ công như GitHub Copilot có thể gây tốn kém thời gian. Khi đó, việc tích hợp trực tiếp các mô hình ngôn ngữ (như gpt-4o-mini) qua API là giải pháp tối ưu.

7.4.3.1. Ngữ cảnh ứng dụng (Use Cases)

Phương pháp này đặc biệt hiệu quả trong hai trường hợp:

- Cần tự động hóa (Automation): Tự động tạo tài liệu (Documentation) cho toàn bộ codebase mỗi khi có bản cập nhật mới.
- Phân tích quy mô lớn (Batch Processing): Quét và giải thích hàng loạt hàm (functions) cùng lúc để kiểm tra tính nhất quán của logic nghiệp vụ.

7.4.3.2. Triển khai kỹ thuật

Bằng cách sử dụng thư viện openai, chúng ta có thể gửi cấu trúc Prompt chi tiết (đã thiết lập ở phần 5) kèm theo mã nguồn để nhận về kết quả phân tích chuẩn hóa.

Ví dụ cấu trúc gọi API:

```
import inspect
import os
from openai import OpenAI
from openai.types.chat import ChatCompletion
from dotenv import load_dotenv

from ch7.src.manhattan import get_manhattan_distance

SURROUND = """you are provided with:
1. A Python function enclosed with {{ FUNCTION }}
2. Arguments requirements enclosed with {{ REQUIREMENTS }}."""
SINGLE_TASK = "Your task is to call the function with the data requirements."

REQUIREMENTS = """
1. 5 rows
2. 3 columns
3. floats and integers in cells
"""

def get_user_prompt(func: callable) -> str:
    return f"""
    FUNCTION: {{{{ inspect.getsource(func) }}}}}

    REQUIREMENTS: {{{{ REQUIREMENTS }}}}}

    EXPLANATION:
    """

if __name__ == "__main__":
    client: OpenAI = OpenAI(
        api_key=os.getenv("OPENAI_API_KEY")
```

```

)

system_prompt = f"{SURROUND} {SINGLE_TASK}"
user_prompt = get_user_prompt(get_manhattan_distance)

completion: ChatCompletion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
)

print("Explanation:", completion.choices[0].message.content)

```

Câu trả lời nhận được:

```

→ python -m ch7.prompts.openai_explain
Explanation: 1. **Function's purpose**: The function 'get_manhattan_distance' is designed to calculate the Manhattan distance between two DataFrames. The Manhattan distance, also known as the L1 distance or taxicab distance, is the sum of the absolute differences of their corresponding elements.

2. **Arguments and their types**: The function takes two arguments:
- 'df1': A pandas DataFrame containing numerical data.
- 'df2': Another pandas DataFrame of the same shape and containing numerical data. The function assumes that both DataFrames have the same dimensions.

3. **Step-by-step data flow**:
- First, it computes the element-wise difference between the two DataFrames using 'df1 - df2'.
- It then takes the absolute value of these differences with the '.abs()' method, resulting in a new DataFrame ('element_wise_dist') where each element represents the absolute difference between the corresponding elements in 'df1' and 'df2'.
- Next, the function sums all the elements in the 'element_wise_dist' DataFrame with 'sum().sum()'. The first 'sum()' computes the sum across columns, and the second 'sum()' aggregates those sums into a total.
- Finally, this total sum is converted to a float type using 'astype(float)' and returned.

4. **Output and its types**: The output of the function is a single value representing the Manhattan distance, and it is of type 'np.float64', which is a NumPy floating-point data type.

5. **Potential edge cases**: The function may encounter several edge cases, such as:
- If 'df1' and 'df2' do not have the same shape, it will raise a ValueError due to shape mismatch during the calculation of the difference.
- If either DataFrame is empty, the function will return a distance of 0.0, which can be misleading as a valid output, but is mathematically correct in terms of the operation performed.
- If the DataFrames contain non-numeric data types, operations will fail, resulting in errors during subsequent calculations.

```

Hình 6: Prompt có ngữ cảnh

7.4.3.3. Lợi ích chiến lược

Tính nhất quán: Đảm bảo mọi hàm trong hệ thống đều được giải thích theo một format duy nhất, giúp các kỹ sư mới (Onboarding) tiếp cận dự án nhanh hơn.

Khả năng mở rộng: Có thể tích hợp trực tiếp vào luồng CI/CD (Continuous Integration/Continuous Deployment) để tự động kiểm tra logic hoặc cập nhật tệp README.md sau mỗi lần commit.

7.5. Hiểu non-project files

7.5.1 requirements.txt

Trong một codebase chuyên nghiệp, việc định nghĩa rõ ràng các thư viện hỗ trợ là yêu cầu bắt buộc. Điều này được thực hiện thông qua tệp requirements.txt, giúp hệ thống xác định chính xác các phiên bản cần thiết để khởi chạy ứng dụng.

7.5.1.1. Cấu trúc và Phiên bản (Versioning)

Việc chỉ định phiên bản cụ thể giúp tránh các lỗi tương thích khi thư viện cập nhật các hàm mới (Breaking changes).

Ví dụ thực tế trong dự án:

- pandas==2.2.3: Thư viện cốt lõi dùng để xử lý cấu trúc dữ liệu DataFrame và thực hiện các phép toán trong hàm get_manhattan_distance.
- flask==3.1.0: Framework web đóng vai trò làm nền tảng cho app.py, giúp tiếp nhận các yêu cầu HTTP và trả về kết quả tính toán.

7.5.1.2. Tầm quan trọng của việc xác định Dependencies

Việc quản lý chặt chẽ các thư viện mang lại 3 lợi ích cốt lõi:

1. Tính đóng gói (Portability): Khi kết hợp với Dockerfile, hệ thống sẽ tự động cài đặt đúng các phiên bản này, đảm bảo môi trường phát triển (Local) và môi trường thực tế (Production) hoàn toàn đồng nhất.
2. Xác định phạm vi ứng dụng: Nhìn vào requirements.txt, một lập trình viên có thể hiểu ngay công nghệ chủ đạo của dự án (Ví dụ: Dự án này là một Web Service xử lý dữ liệu).
3. Bảo mật và Bảo trì: Dễ dàng kiểm tra xem các phiên bản đang dùng có lỗ hổng bảo mật nào không và lên kế hoạch nâng cấp định kỳ.

7.5.2. Dockerfile

Nội dung Dockerfile:

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 5000s

CMD ["python", "app.py"]
```

Giải thích:

Dưới đây là các chỉ thị cốt lõi được sử dụng để xây dựng Image cho ứng dụng:

- FROM python:3.10-slim: Lựa chọn hệ điều hành nền tảng nhẹ (slim) đã cài đặt sẵn Python 3.10 để tối ưu dung lượng.
- WORKDIR /app: Thiết lập thư mục làm việc chính bên trong container.
- COPY requirements.txt . & RUN pip install...: Tách riêng bước cài đặt thư viện để tận dụng cơ chế Layer Caching của Docker, giúp tăng tốc độ build ở những lần sau.
- COPY . .: Sao chép toàn bộ mã nguồn (bao gồm app.py và thư mục src/) vào container.

Hai chỉ thị cuối cùng đóng vai trò quyết định cách ứng dụng giao tiếp với thế giới bên ngoài:

- EXPOSE 5000: Đây là một chỉ thị mang tính tài liệu. Nó xác định cổng (port) mà ứng dụng dự định sẽ lắng nghe (trong trường hợp này là cổng 5000 của Flask).

Lưu ý kỹ thuật: Lệnh này không tự động mở port trên máy chủ; nó đóng vai trò thông báo cho người vận hành biết cần phải "map" port nào khi chạy container.

- CMD ["python", "app.py"]: Định nghĩa lệnh thực thi mặc định. Khi container được khởi động, đây là lệnh duy nhất và cuối cùng được chạy để kích hoạt Entry point của ứng dụng.

7.6. Debugging – Cách hiểu code hiệu quả nhất:

Việc đọc mã nguồn thuần túy thường khó giúp chúng ta hình dung được sự thay đổi của dữ liệu qua từng bước. Do đó, quy trình thực nghiệm dưới đây được khuyến nghị áp dụng để nắm bắt logic hệ thống một cách triệt để.

7.6.1. Quy trình thực nghiệm 3 bước

Để hiểu rõ hàm `get_manhattan_distance`, kỹ sư không nên chỉ xem code mà nên thực hiện:

- Khởi tạo dữ liệu mẫu (Mock Data): Tạo các bảng dữ liệu nhỏ (DataFrame) với các giá trị đã biết trước để dễ dàng đối chiếu kết quả.
- Chạy Debug (Debugging): Sử dụng các công cụ debug trong IDE để dừng chương trình tại từng dòng lệnh.

Ví dụ lệnh thực thi: $d = \text{get_manhattan_distance}(df1, df2)$

- Quan sát biến (Variable Inspection): Theo dõi giá trị của các biến trong bộ nhớ tại thời điểm thực thi.

7.6.2. Giá trị của việc Debug trong phân tích hệ thống

Hành động Debug mang lại hai giá trị cốt lõi mà việc đọc code thông thường không thể cung cấp:

- Minh bạch hóa luồng dữ liệu (Data Flow): Giúp xác định chính xác cách dữ liệu biến đổi từ một DataFrame phức tạp thành một giá trị vô hướng (scalar).

- Kiểm soát giá trị trung gian: Cho phép ta quan sát biến `element_wise_dist` sau khi lấy trị tuyệt đối nhưng trước khi tính tổng. Đây là chìa khóa để phát hiện ra các lỗi logic tiềm ẩn hoặc các giá trị bất thường (như NaN hoặc Inf) mà mắt thường dễ bỏ sót.

7.7. Tạo dữ liệu mẫu bằng AI

Việc chạy Debug và quan sát biến chỉ thực sự hiệu quả khi chúng ta có một tập dữ liệu mẫu (Mock Data) đủ tốt. Sự kết hợp giữa GitHub Copilot và ChatGPT giúp lập trình viên nhanh chóng tạo ra các bộ dữ liệu mô phỏng sát với thực tế.

7.7.1. Sử dụng GitHub Copilot để tự sinh mã nguồn (Auto-generation)

GitHub Copilot thể hiện thế mạnh trong việc dự đoán ngữ cảnh. Khi bạn bắt đầu viết hàm kiểm thử, Copilot có thể:

- Tự sinh cấu trúc DataFrame: Chỉ cần gõ tên biến `df_test`, AI sẽ gợi ý các dòng mã tạo dữ liệu mẫu với thư viện Pandas.
- Đề xuất giá trị ngẫu nhiên: Sử dụng `numpy.random` để tạo ra các ma trận số thực phù hợp với đầu vào của hàm Manhattan.

7.7.2. Tùy chỉnh dữ liệu chuyên sâu với ChatGPT

Trong khi Copilot mạnh về tốc độ viết code, ChatGPT lại vượt trội trong việc tùy chỉnh kịch bản dữ liệu (Data Customization). Bạn có thể yêu cầu ChatGPT thiết kế các bộ dữ liệu theo những thông số kỹ thuật cụ thể để kiểm tra tính chịu tải và độ bền của hàm:

- Điều chỉnh quy mô: Yêu cầu tạo dữ liệu với số dòng và số cột cực lớn để kiểm tra hiệu năng tính toán.
- Kiểm soát kiểu dữ liệu: Yêu cầu xen kẽ các cột dữ liệu số (Integer, Float) với dữ liệu chuỗi (String) để kiểm thử khả năng xử lý lỗi của hệ thống.
- Mô phỏng dữ liệu nhiễu: Tạo các DataFrame chứa giá trị rỗng (NaN) hoặc giá trị ngoại lai (Outliers) để quan sát cách hàm `get_manhattan_distance` phản ứng trong các trường hợp biên.

7.7.3. Lợi ích của việc kết hợp AI trong Testing

Sự phối hợp này tạo ra một quy trình kiểm thử khép kín:

- ChatGPT thiết kế kịch bản dữ liệu phức tạp.
- Copilot thực thi mã nguồn khởi tạo nhanh chóng trên IDE.
- Lập trình viên thực hiện Debug để quan sát giá trị trung gian.

7.8. So sánh công cụ

Công cụ	Ưu điểm	Nhược điểm
Copilot	Nhanh	Ít tùy chỉnh
ChatGPT	Linh hoạt	Cần viết prompt
OpenAI API	Tự động hóa	Phức tạp

Bảng 2: Bảng so sánh các công cụ AI

7.9. Kết luận

7.9.1. Thay đổi tư duy tiếp cận mã nguồn

Việc hiểu mã nguồn không còn đơn thuần là hoạt động "đọc hiểu" tĩnh. Trong môi trường phát triển phức tạp, sự hiểu biết thực sự đến từ việc tương tác chủ động với hệ thống. Chúng ta cần nhìn nhận codebase như một thực thể sống, nơi dữ liệu biến đổi liên tục qua các tầng xử lý.

7.9.2. Vai trò chiến lược của Generative AI (GenAI)

GenAI (GitHub Copilot, ChatGPT) không chỉ là công cụ viết code tự động mà còn là một trợ lý phân tích mạnh mẽ.

- Giải thích: Giúp làm rõ các hàm phức tạp như `get_manhattan_distance`.
- Khởi tạo: Tự động sinh dữ liệu mẫu (DataFrame) với các thông số tùy chỉnh về số dòng, số cột và kiểu dữ liệu để phục vụ kiểm thử. Tuy nhiên, AI chỉ hiệu quả khi được định hướng bởi các Prompt chi tiết và có cấu trúc.

7.9.3. Debugging: Kỹ năng định hình năng lực kỹ sư

Dù các công cụ hỗ trợ có phát triển đến đâu, Debugging vẫn là kỹ năng quan trọng nhất. Khả năng đặt Breakpoint, quan sát các biến trung gian và theo dõi luồng dữ liệu (Data flow) thực tế là cách duy nhất để kiểm chứng những gì AI giải thích và đảm bảo hệ thống vận hành chính xác trong mọi trường hợp biên (Edge cases).

7.9.4. Mô hình phân tích hiện đại

Một quy trình phân tích mã nguồn tối ưu hiện nay cần sự kết hợp chặt chẽ giữa ba trụ cột:

- AI: Cung cấp cái nhìn tổng quan và giải thích nhanh logic.
- Testing: Sử dụng dữ liệu mẫu để mô phỏng các kịch bản thực tế.
- Debugging: Quan sát thực tế để xác nhận kết quả và phát hiện sai sót.

Tổng kết: Sự giao thoa giữa trí tuệ nhân tạo và kỹ năng thực nghiệm không chỉ giúp lập trình viên hiểu code nhanh hơn mà còn đảm bảo độ chính xác và tính bền vững cho toàn bộ dự án phần mềm.

Chương 8: GIỚI THIỆU VỀ PROMPT ENGINEERING

8.1. Giới thiệu

Chương 8 của sách *Supercharged Coding with GenAI* tập trung vào việc giới thiệu Prompt Engineering – một lĩnh vực quan trọng trong việc làm việc với các mô hình ngôn ngữ lớn (LLMs) như ChatGPT, GitHub Copilot và OpenAI API.

Prompt Engineering giúp lập trình viên:

- kiểm soát output của AI
- tạo ra code chính xác hơn
- đảm bảo tính nhất quán trong coding style

8.2. Few-shot Learning

8.2.1 Khái niệm

Few-shot learning (hay còn gọi là in-context learning) là kỹ thuật cung cấp cho mô hình một số ví dụ đầu vào và đầu ra để hướng dẫn cách xử lý dữ liệu mới.

Thay vì chỉ mô tả yêu cầu bằng ngôn ngữ tự nhiên, ta đưa trực tiếp các ví dụ:

```
INPUT_1 → OUTPUT_1  
INPUT_2 → OUTPUT_2  
INPUT → ?
```

Điều này giúp mô hình:

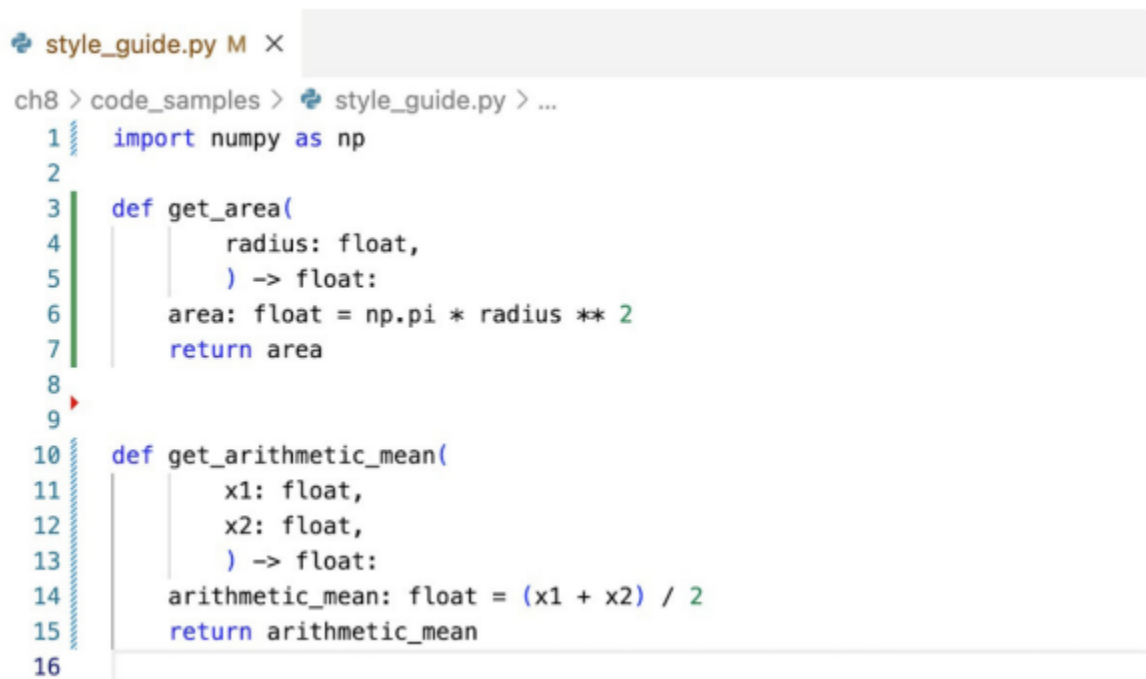
- hiểu rõ format đầu ra
- học được pattern
- giảm sai sót

8.2.2. Ứng dụng Few-shot Learning trong lập trình

Few-shot learning là một kỹ thuật quan trọng trong Prompt Engineering, cho phép mô hình học cách xử lý dữ liệu thông qua một số lượng nhỏ các ví dụ được cung cấp trực tiếp trong prompt. Trong lĩnh vực lập trình, kỹ thuật này được ứng dụng rộng rãi nhằm kiểm soát cách sinh mã nguồn của mô hình, đảm bảo tính nhất quán và phù hợp với yêu cầu cụ thể của hệ thống.

8.2.2.1. Chuẩn hóa coding style

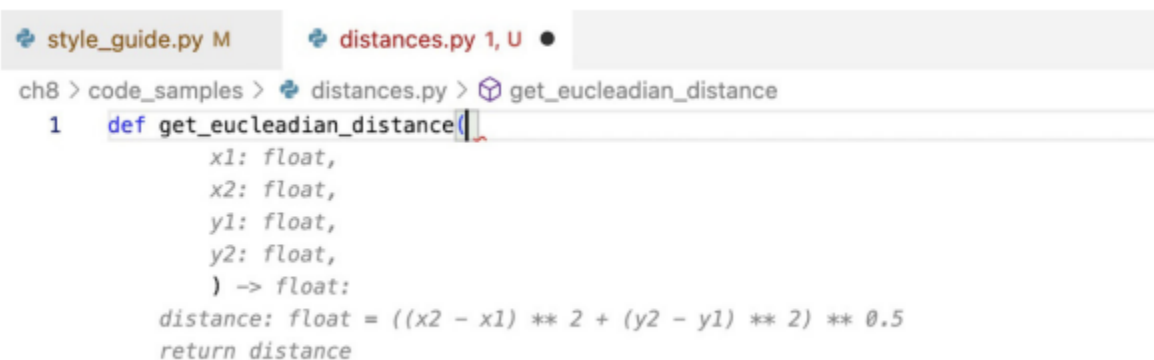
Một trong những ứng dụng phổ biến của few-shot learning là chuẩn hóa phong cách lập trình. Trong thực tế, mã nguồn do mô hình sinh ra có thể không tuân thủ quy ước đặt tên, kiểu dữ liệu hoặc cấu trúc hàm của dự án. Để khắc phục, ta cung cấp các ví dụ minh họa về cách chuyển đổi từ mã nguồn chưa chuẩn sang mã nguồn đã được định dạng.



```
1 import numpy as np
2
3 def get_area(
4     radius: float,
5 ) -> float:
6     area: float = np.pi * radius ** 2
7     return area
8
9
10 def get_arithmetic_mean(
11     x1: float,
12     x2: float,
13 ) -> float:
14     arithmetic_mean: float = (x1 + x2) / 2
15     return arithmetic_mean
16
```

Hình 7: Chuẩn hóa Coding style

Khi áp dụng mô hình, kết quả thu được:



```
1 def get_euclidean_distance(
2     x1: float,
3     x2: float,
4     y1: float,
5     y2: float,
6 ) -> float:
7     distance: float = ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5
8     return distance
```

Hình 8: Mô hình học quy tắc coding style

Qua đó có thể thấy mô hình đã học được quy tắc định dạng và áp dụng nhất quán cho dữ liệu đầu vào mới.

8.2.2.2. Chuẩn hóa logging và comment

Trong quá trình phát triển phần mềm, việc ghi log và viết comment theo chuẩn là rất quan trọng. Few-shot learning có thể được sử dụng để đảm bảo các đoạn mã sinh ra tuân thủ định dạng logging thống nhất.

Ví dụ:

```
CONTEXT: You are provided with:
1. Python snippet enclosed with {{{ OLD }}}
2. Examples enclosed with {{{ OLD_i }}} followed by a desirable output
enclosed with {{{ REFACTORED_i }}}

TASK: Refactor the snippet to a log message.

OLD_1: {{{ print('Process started for config.txt with verbose=True') }}}
REFACTORED_1: {{{ logger.info('Processing started', extra={'verbose':
True, file: 'config.txt'}) }}}

OLD_2: {{{ print('Warning! Could not load user U-232 data from user_info.
csv') }}}
REFACTORED_2: {{{ logger.warning('User data failed to load',
extra={'user': 'U-232', 'file': 'user_info.csv'}) }}}

OLD: {{{ print('Error! File not found: passwords.txt') }}}
REFACTORED: Hila Paz Herszfang, 9 months ago • ch8
```

Hình 9: Chuẩn hóa logging và comment

Nhờ đó, hệ thống có thể duy trì được cấu trúc logging đồng nhất mà không cần định nghĩa quy tắc phức tạp.

8.2.2.3. Đánh giá

Few-shot learning mang lại nhiều lợi ích trong lập trình:

- không yêu cầu huấn luyện lại mô hình
- dễ triển khai và linh hoạt
- cải thiện đáng kể chất lượng mã nguồn sinh ra

Tuy nhiên, hiệu quả của phương pháp phụ thuộc vào chất lượng và độ đại diện của các ví dụ được cung cấp.

8.3. Iterative Prompting

Iterative prompting là kỹ thuật tương tác lặp lại với mô hình trong một vòng phản hồi (feedback loop), trong đó đầu ra sẽ được cải thiện dần dựa trên việc đánh giá các phản hồi trước đó.

Phản hồi này có thể:

- Thủ công: ví dụ yêu cầu ChatGPT sửa lại câu trả lời để khắc phục lỗi chưa xác định
- Tự động: ví dụ compile code do API tạo ra hoặc chạy qua unit test

Kết quả phản hồi (như lỗi traceback hoặc test fail) sẽ được đưa ngược lại vào mô hình, và quá trình này lặp lại liên tục cho đến khi đạt được tiêu chí mong muốn.

```
for i in range(100):
    try:
        exec(user_code)
        print(f"successfully compiled:\n {user_code}")
        break

    except SyntaxError as se:
        user_code = get_refined_output(user_code, traceback.format_exc())
```

Hình 10: Ví dụ về Iterative Programming

Trong ví dụ trên, chương trình thực hiện các bước như sau:

1. Sử dụng vòng lặp để thử thực thi đoạn mã `user_code` tối đa 10 lần
2. Nếu mã thực thi thành công, chương trình in ra kết quả và kết thúc vòng lặp
3. Nếu xảy ra lỗi cú pháp (`SyntaxError`), hệ thống sẽ:
 - Thu thập thông tin lỗi thông qua `traceback.format_exc()`
 - Gửi lại lỗi này cùng với mã hiện tại vào mô hình thông qua hàm `get_refined_output`
 - Nhận về phiên bản mã đã được chỉnh sửa và tiếp tục thử lại

Cách tiếp cận này cho phép mô hình từng bước cải thiện mã nguồn dựa trên phản hồi thực tế, thay vì chỉ tạo ra một kết quả duy nhất ngay từ đầu.

Ngoài việc kiểm tra lỗi biên dịch, phương pháp này còn có thể được mở rộng bằng cách:

- Thử thi mã với bộ kiểm thử (test suite)
- Đánh giá dựa trên các tiêu chí chất lượng khác

Nhờ đó, iterative prompting trở thành một kỹ thuật hiệu quả trong việc kết hợp trí tuệ nhân tạo với các quy trình kiểm thử tự động, giúp nâng cao độ chính xác và độ tin cậy của mã nguồn được sinh ra.

8.4 Template-based Prompting

Template-based prompting là kỹ thuật định nghĩa trước cấu trúc output, thay vì chỉ đưa ví dụ.

Ví dụ template:

```
CONTEXT: You will be provided with Python code enclosed with {{{ CODE }}} that does not compile, and an error
        traceback enclosed with {{{ TRACEBACK }}}}, along with a template of the output enclosed with {{{ TEMPLATE }}}.

TASK: Fix the errors in the code using the provided template.

TEMPLATE: This code requires a fix because of { ERROR TYPE }. The fixed code is { CODE }.
```

Hình 11: Template-based Prompting

Lợi ích:

- dễ tái sử dụng
- chuẩn hóa prompt
- kiểm soát output chặt chẽ

8.5. Chain-of-Thought Prompting (CoT)

Chain-of-Thought (CoT) prompting là kỹ thuật hướng dẫn mô hình suy luận theo từng bước (step-by-step) để đi đến kết quả cuối cùng. Thay vì yêu cầu mô hình đưa ra đáp án ngay lập tức, phương pháp này giúp mô hình phân tách bài toán thành các bước nhỏ, từ đó cải thiện độ chính xác và tính logic của kết quả.

Ví dụ thay vì viết code trực tiếp, ta chia thành các bước:

```
def validate_no_bugs_in_source_code(sc: str) -> None:
    validate_syntax(sc)
    validate_compilation(sc)
    validate_reproducibility_across_runs(sc)
```

Hình 12: Ví dụ về Chain-of-Thought Prompting

Trong đó:

- `validate_no_bugs_in_source_code` đóng vai trò định nghĩa luồng xử lý tổng thể
- Các hàm như `validate_syntax`, `validate_compilation`, `validate_reproducibility_across_runs` đại diện cho các bước kiểm tra cụ thể
- Tuy nhiên, các hàm này chưa cần cài đặt ngay

Cách thiết kế này cho phép:

- Tách biệt rõ ràng giữa logic tổng thể và chi tiết triển khai
- Tận dụng khả năng của GenAI trong việc tự động sinh phần cài đặt cấp thấp

Khi bắt đầu triển khai từng hàm, ví dụ như `validate_syntax`, công cụ như GitHub Copilot có thể tự động đề xuất phần cài đặt phù hợp. Minh họa trong Hình 8.7 cho thấy Copilot đã sinh mã sử dụng thư viện `ast` để kiểm tra cú pháp:

- Sử dụng `ast.parse(sc)` để phân tích cú pháp
- Bắt lỗi `SyntaxError` nếu có
- Ném ra ngoại lệ (`ValueError`) khi phát hiện lỗi cú pháp

Cách tiếp cận này cho thấy CoT không chỉ áp dụng trong việc suy luận mà còn rất hiệu quả trong thiết kế và sinh mã có cấu trúc, giúp:

- Tăng tính rõ ràng của chương trình
- Dễ kiểm soát và mở rộng
- Tận dụng tối đa khả năng hỗ trợ của các công cụ GenAI

8.6 Kết luận

Chương này đã giới thiệu few-shot learning như một kỹ thuật Prompt Engineering giúp định hướng các mô hình ngôn ngữ lớn tạo ra kết quả chính xác và nhất quán hơn thông qua các ví dụ minh họa. Kỹ thuật này được áp dụng hiệu quả trong các bài toán như chuẩn hóa coding style và logging, đồng thời có thể triển khai trên nhiều nền tảng như ChatGPT, OpenAI API và GitHub Copilot.

Bên cạnh đó, chương cũng trình bày các kỹ thuật quan trọng khác như iterative prompting, template-based prompting và chain-of-thought prompting, giúp cải thiện chất lượng và cấu trúc của mã nguồn sinh ra. Đây là nền tảng để tiếp tục nghiên cứu các phương pháp nâng cao trong các chương tiếp theo.

CHƯƠNG 9: KỸ THUẬT PROMPT NÂNG CAO CHO CÁC TÁC VỤ LIÊN QUAN ĐẾN MÃ NGUỒN

9.1 Giới thiệu

Trong Chapter 9, Herszfang & Henstock (2025) mở rộng các kỹ thuật prompt engineering sang các bài toán lập trình phức tạp hơn. Trọng tâm của chương là:

- đánh giá baseline outputs
- cải thiện kết quả bằng Chain-of-Thought (CoT)
- sử dụng prompt chaining

Một ví dụ xuyên suốt trong chương là bài toán tính geometric mean, được dùng để so sánh hiệu quả của các phương pháp prompting khác nhau.

9.2. Mở rộng phạm vi thực thi của Prompt (Extending Prompt Scope)

Trong thực tế, các bài toán lập trình thường yêu cầu xử lý logic đa bước thay vì chỉ một công thức toán học duy nhất. Herszfang & Henstock (2025) lấy ví dụ về việc mở rộng bài toán tính trung bình nhân (geometric mean) để xử lý các danh mục đầu tư có lợi nhuận âm.

Phân tích: Một lỗi phổ biến của các Prompt cơ bản là không xử lý được các trường hợp biên hoặc dữ liệu không hợp lệ. Việc mở rộng phạm vi yêu cầu lập trình viên phải cung cấp ngữ cảnh về các quy tắc nghiệp vụ phức tạp, ví dụ như chuyển đổi lợi nhuận ròng (net returns) sang lợi nhuận gộp (gross returns) trước khi tính toán để tránh lỗi toán học khi gặp số âm.

9.3 Baseline Prompt và Kết quả ban đầu

GitHub Copilot Baseline

Cách tiếp cận: chỉ cần cung cấp function signature.

```
def get_geometric_mean(
    net_returns: Dict[str, float],
) -> float:
    product: float = 1
    for key in net_returns:
        product *= net_returns[key]
    geometric_mean: float = product ** (1 / len(net_returns))
    return geometric_mean
```

Hình 13: Github Copilot Baseline

Ưu điểm:

- Nhanh, đơn giản
- Sinh code gọn, ít dư thừa

Nhược điểm:

- Không xử lý tốt các trường hợp phức tạp (ví dụ: input âm)

ChatGPT Baseline

Ý tưởng chính:

ChatGPT cần prompt rõ ràng hơn để sinh code đúng mong muốn.

Prompt minh họa:

```
CONTEXT: You are provided with a Python function signature enclosed with {{{ FUNCTION }}}.
TASK: implement the function.

FUNCTION: {{{ def get_geometric_mean(net_returns: Dict[str, float]) -> float: }}}

CODE: Hila Paz Herszfang, 9 months ago • ch9 ...
```

Hình 14: ChatGPT Baseline

Output

```
def get_geometric_mean(net_returns: Dict[str, float]) -> float:
    """ [... omitted for brevity ...] """
    if not net_returns:
        return 0.0
    # Calculate the product of (1 + return) for each net return
    product = math.prod(1 + r for r in net_returns.values())
    n = len(net_returns)
    # Compute the nth root of the product and subtract 1 to get the
    geometric mean
    geometric_mean = product ** (1 / n) - 1
    return geometric_mean
```

Hình 15: Output của ChatGPT Baseline

Ưu điểm:

- Code đúng về mặt logic
- Xử lý được bài toán tốt hơn Copilot

Nhược điểm:

- Sinh ra nhiều phần không cần thiết:
- Docstring
- Comment dư thừa
- Input validation
- Format không tối ưu

OpenAI API Baseline

Ý tưởng chính:

Sử dụng API cho kết quả tương tự ChatGPT vì cùng model.

Prompt

```
SURROUND = "You are provided with a Python function signature enclosed with {{{ FUNCTION }}}."
SINGLE_TASK = "Your task is to implement the function."

SRC_CODE = """def get_geometric_mean(\n\t net_returns: Dict[str, float],\n) -> float:"""

def get_user_prompt(src: str) -> str:
    return f"""
    FUNCTION: {{{{{{ {src} }}}}}}

    CODE:
    """
```

Hình 16: OpenAI API Baseline

Output

```
def get_geometric_mean(net_returns: Dict[str, float]) -> float:
    # Return 0.0 if there are no returns (could alternatively raise an
    exception)
    if not net_returns:
        return 0.0
    import math

    # Calculate the product of (1 + return) for each period
    product = 1.0
    n = len(net_returns)
    for r in net_returns.values():
        product *= (1 + r)
```

```
# Compute the geometric mean: nth root of product then subtract 1
geom_mean = math.pow(product, 1 / n) - 1
return geom_mean
```

Hình 17: Output của OpenAI API Baseline

Ưu điểm:

- Linh hoạt, dễ tích hợp hệ thống
- Kết quả chính xác

Nhược điểm:

- Vẫn gặp các vấn đề giống ChatGPT:
- Comment dư
- Import sai vị trí
- Validation không cần thiết

Kết luận:

- Copilot: nhanh nhưng thiếu chính xác trong case phức tạp
- ChatGPT/API: chính xác hơn nhưng cần prompt tốt để tránh code “rác”
- Khi bài toán phức tạp → cần prompt engineering nâng cao (CoT, chaining)

9.4. Kỹ thuật Chain-of-Thought (CoT) trong lập trình

Chain-of-Thought (Chuỗi suy nghĩ) là kỹ thuật hướng dẫn mô hình thực hiện các bước lập luận trung gian trước khi đưa ra câu trả lời cuối cùng. Chương 9 của cuốn sách khẳng định rằng CoT giúp giảm thiểu hiện tượng "ảo giác" (hallucination) trong lập trình.

Phân tích: Thay vì yêu cầu LLM "viết mã ngay lập tức", lập trình viên yêu cầu mô hình "giải thích logic từng bước". Điều này đặc biệt hiệu quả với các mô hình nhỏ hơn như GPT-4o-mini, giúp chúng đạt được hiệu suất tương đương các mô hình lớn hơn khi xử lý logic phức tạp (Herszfang & Henstock, 2025).

9.5 Sử dụng CoT (Chain of Thought) với GitHub Copilot

Ý tưởng chính:

CoT (Chain of Thought) là kỹ thuật chia nhỏ bài toán thành các bước logic rõ ràng, giúp Copilot sinh code dễ hiểu, sạch hơn và chính xác hơn.

Thay vì yêu cầu viết một hàm phức tạp ngay lập tức, ta hướng dẫn Copilot theo từng bước.

Đối với bài toán tính trung bình hình học (geometric mean) từ tập net returns, ta có thể chia thành ba bước:

1. Chuyển đổi từ net returns sang gross returns
2. Tính geometric mean của gross returns
3. Chuyển đổi kết quả từ gross returns về net returns

Cách tiếp cận này được thể hiện thông qua hàm tổng như sau:

```
def get_average_return(  
    net_returns: Dict[str, float],  
    ) -> float:  
    gross_returns: np.ndarray = get_gross_returns(net_returns)  
    gross_average: float = get_geometric_mean(gross_returns)  
    net_average: float = get_net_average(gross_average)  
    return net_average
```

Hình 18: Sử dụng CoT (Chain of Thought) với GitHub Copilot

Tại thời điểm này, các hàm như `get_gross_returns`, `get_geometric_mean`, và `get_net_average` chưa được cài đặt, do đó IDE có thể hiển thị cảnh báo lỗi. Tuy nhiên, đây là hành vi bình thường và không ảnh hưởng đến quá trình phát triển.

Dựa trên cấu trúc đã định nghĩa, Copilot sẽ:

- Tự động suy luận các bước cần thiết
- Sinh code cho từng hàm theo đúng thứ tự được gọi
- Hiểu được kiểu dữ liệu đầu vào và đầu ra thông qua type hints

Ví dụ, sau khi định nghĩa hàm tổng, Copilot sẽ lần lượt đề xuất cài đặt cho các hàm thành phần:

```
def get_gross_returns(
    net_returns: Dict[str, float],
) -> np.ndarray:
    gross_returns: np.ndarray = np.array(list(net_returns.values())) + 1
    return gross_returns

def get_geometric_mean(
    gross_returns: np.ndarray,
) -> float:
    gross_average: float = np.prod(gross_returns) ** (1 / len(gross_returns))
    return gross_average

def get_net_average(
    gross_average: float,
) -> float:
    net_average: float = gross_average - 1
    return net_average
```

Hình 19: Copilot đề xuất cài đặt cho các hàm thành phần

9.6. Áp dụng Chain of Thought (CoT) với ChatGPT và OpenAI API

9.6.1. CoT với ChatGPT

Tương tự như GitHub Copilot, kỹ thuật Chain of Thought (CoT) cũng có thể được áp dụng hiệu quả với ChatGPT trong các bài toán lập trình. Thay vì chỉ cung cấp chữ ký hàm, ta cung cấp toàn bộ hàm tổng (high-level function) và yêu cầu model hoàn thiện các hàm con còn thiếu.

Cách tiếp cận này giúp mô hình hiểu rõ quy trình xử lý và sinh code chính xác hơn.

Prompt minh họa:

```
CONTEXT: You are provided with a Python function enclosed with {{{ FUNCTION }}} that calls functions that should be completed.
TASK: Implement the missing functions.
FUNCTION: {{{def get_average_return(
    net_returns: Dict[str, float],
) -> float:
    gross_returns: np.ndarray = get_gross_returns(net_returns)
    gross_average: float = get_geometric_mean(gross_returns)
    net_average: float = get_net_average(gross_average)
    return net_average }}}
CODE: Hila Paz Herszfang, 9 months ago • ch9
```

Hình 20: CoT với ChatGPT

Output

```
def get_gross_returns(net_returns: Dict[str, float]) -> np.ndarray:
    """ ... omitted for brevity ... """
    return np.array([1 + net_return for net_return in net_returns.
values()])

def get_geometric_mean(values: np.ndarray) -> float:
    """ ... omitted for brevity ... """
    return np.prod(values) ** (1 / len(values))

def get_net_average(gross_average: float) -> float:
    """ ... omitted for brevity ... """
    return gross_average - 1

def get_average_return(net_returns: Dict[str, float]) -> float:
    gross_returns: np.ndarray = get_gross_returns(net_returns)
    gross_average: float = get_geometric_mean(gross_returns)

net_average: float = get_net_average(gross_average)
return net_average
```

Hình 21: Kết quả đầu ra của CoT với ChatGPT

Đánh giá

1. So với phương pháp baseline, kết quả từ CoT với ChatGPT có nhiều cải thiện:
2. Không còn:
 - Comment dư thừa
 - Input validation không cần thiết
3. Giữ được cấu trúc rõ ràng theo từng bước
4. Giảm đáng kể công sức chỉnh sửa thủ công

Đáng chú ý, kết quả này đạt được ngay cả khi sử dụng mô hình GPT-4o mini, cho thấy hiệu quả của prompt quan trọng hơn việc sử dụng model mạnh.

9.6.2. CoT với OpenAI API

Kỹ thuật CoT cũng có thể áp dụng tương tự khi sử dụng OpenAI API. Ta cung cấp hàm tổng và yêu cầu model hoàn thiện các hàm còn thiếu thông qua system prompt và user prompt.

Prompt minh họa


```

SURROUND = "You are provided with a Python function enclosed with {{{ FUNCTION }}} that calls functions that should be completed."
SINGLE_TASK = "Your task is to implement the missing functions."

def get_user_prompt(func: callable) -> str:
    return f"""
FUNCTION: {{{{ inspect.getsource(func) }}}}}
CODE:
"""
    You, now * Uncommitted changes

```

Hình 22: CoT với OpenAI API

Output

```

def get_gross_returns(net_returns: Dict[str, float]) -> np.ndarray:
    """Convert net returns to gross returns."""
    return np.array([1 + r for r in net_returns.values()])

def get_geometric_mean(gross_returns: np.ndarray) -> float:
    """Calculate the geometric mean of a numpy array of gross returns."""
    return np.prod(gross_returns) ** (1 / len(gross_returns))

def get_net_average(gross_average: float) -> float:
    """Convert gross average back to net average."""
    return gross_average - 1

def get_average_return(net_returns: Dict[str, float]) -> float:
    ... omitted for brevity ...

```

Hình 23: Kết quả đầu ra của CoT với OpenAI API

Đánh giá

1. Kết quả tốt hơn baseline
2. Giữ được mức trừu tượng hợp lý
3. Tránh được:
 - Inline comments
 - Input validation không cần thiết
4. Tuy nhiên, trong một số trường hợp, CoT vẫn chưa đủ để tạo ra code hoàn chỉnh ngay lập tức.

Kết luận

- CoT giúp cải thiện đáng kể chất lượng code khi sử dụng ChatGPT và OpenAI API
- Prompt rõ ràng giúp model hiểu đúng luồng xử lý

- Tuy nhiên, để đạt code “production-ready”, cần kết hợp thêm kỹ thuật chaining (prompt nhiều bước)

9.7. Sử dụng Prompt Chaining với các mô hình LLM

9.7.1. Giới thiệu

Prompt Chaining là một kỹ thuật quan trọng trong prompt engineering, cho phép cải thiện chất lượng đầu ra của các mô hình ngôn ngữ lớn (LLMs) bằng cách chia nhỏ bài toán thành nhiều bước xử lý tuần tự.

Thay vì yêu cầu mô hình tạo ra kết quả hoàn chỉnh ngay từ đầu, Prompt Chaining hướng đến việc:

- Giải quyết từng phần nhỏ của bài toán
- Tinh chỉnh dần kết quả qua nhiều lần tương tác

Cách tiếp cận này giúp mô hình hoạt động hiệu quả hơn và tạo ra kết quả có chất lượng cao hơn.

9.7.2 Áp dụng Prompt Chaining với ChatGPT

Việc áp dụng Prompt Chaining với ChatGPT khá trực quan nhờ giao diện hội thoại. Lịch sử trò chuyện cho phép người dùng tận dụng lại các prompt trước đó mà không cần nhập lại toàn bộ ngữ cảnh.

Tuy nhiên, nếu cố gắng yêu cầu quá nhiều nhiệm vụ trong một prompt, kết quả thường không tối ưu. Do đó, cần tuân thủ nguyên tắc mỗi prompt chỉ nên tập trung vào một nhiệm vụ (single-task).

Khi áp dụng Prompt Chaining với ChatGPT, có hai nguyên tắc quan trọng:

- Cùng ngữ cảnh:
→ Chỉ cần đưa ra yêu cầu mới (không cần lặp lại toàn bộ prompt)
- Khác ngữ cảnh (ví dụ: viết test):
→ Cần viết lại prompt đầy đủ theo các best practices (như quy tắc five S’s)

Đối với bài toán `get_average_return`, quá trình chaining có thể thực hiện theo các bước:

- Prompt 1: Sinh code hoàn chỉnh sử dụng CoT
- Prompt 2: Thêm type hints cho tất cả biến
- Prompt 3: Chuẩn hóa docstring theo Google Style
- Prompt 4: Viết unit test (tạo ngữ cảnh mới)

Ví dụ minh họa

Prompt 2 – Thêm type hints:

Add type hints to all variables.

→ **Kết quả:**

- Các biến như `gross_returns` được annotate rõ ràng (`np.ndarray`)
- Code trở nên chặt chẽ và dễ hiểu hơn

Prompt 3 – Chuẩn hóa docstring:

Include Google Style docstring.

→ **Kết quả:**

1. Docstring được chuẩn hóa theo format Google
2. Bao gồm đầy đủ:
 - Args
 - Returns

Ví dụ:

```
def get_geometric_mean(gross_returns: np.ndarray) -> float:
    """Calculates the geometric mean of gross returns.
    Args:
        gross_returns (np.ndarray): An array of gross returns.
    Returns:
        float: The geometric mean of the gross returns.
    """
    ... omitted for brevity ...

def get_net_average(gross_average: float) -> float: """Converts gross
average return to net average return.
Args:
    gross_average (float): The geometric mean of gross returns.
```

Hình 24: Chuẩn hóa docstring

```

Returns:
    float: The corresponding net average return.
"""
... omitted for brevity ...

def get_average_return(net_returns: Dict[str, float]) -> float:
    """Calculates the geometric mean of net returns from a dictionary of net
    returns.
    Args:
        net_returns (Dict[str, float]): A dictionary of net returns per
        period.

    Returns:
        float: The geometric mean of the net returns.
    """
    ... omitted for brevity ...

```

Hình 25: Chuẩn hóa docstring (v2)

Prompt Chaining với ChatGPT là một phương pháp hiệu quả để cải thiện chất lượng code theo từng bước nhỏ. Bằng cách tách các yêu cầu thành nhiều prompt đơn nhiệm, người dùng có thể kiểm soát tốt hơn quá trình sinh code và đạt được kết quả gần với production-ready.

Tuy nhiên, để có quyền kiểm soát cao hơn đối với prompt và context, việc sử dụng OpenAI API sẽ là một giải pháp phù hợp hơn, sẽ được trình bày ở phần tiếp theo.

9.7.3 Tối ưu Prompt Chaining với Selective History trong OpenAI API

Khi áp dụng Prompt Chaining với OpenAI API, một vấn đề quan trọng phát sinh là độ dài prompt tăng dần theo từng bước, do toàn bộ lịch sử hội thoại được giữ lại. Điều này dẫn đến:

- Tăng số lượng token sử dụng (chi phí cao hơn)
- Làm mô hình bị phân tán khỏi nhiệm vụ chính

Để giải quyết vấn đề này, kỹ thuật Selective History được đề xuất, cho phép chỉ giữ lại những thông tin cần thiết trong mỗi bước.

Trong cách tiếp cận cơ bản, toàn bộ lịch sử prompt và phản hồi được lưu lại:

```
messages = [{"role": "system", "content": system_prompt}]

prompt_1 = get_user_prompt(get_average_return)
```

```
prompt_2 = "Add type hints to all variables."
prompt_3 = "include Google Style docstring."

for prompt in [prompt_1, prompt_2, prompt_3]:
    messages.append({"role": "user", "content": prompt})

completion: ChatCompletion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
)

output: str = completion.choices[0].message.content
messages.append({"role": "assistant", "content": output})
```

Hình 26: Lưu lại lịch sử Prompt và phản hồi

Hạn chế:

- Token tăng nhanh (ví dụ: 1,253 tokens ở bước 3)
- Không tối ưu cho chuỗi prompt dài

Giải pháp Selective History

Thay vì truyền toàn bộ lịch sử, ta chỉ:

Lấy phần code cần thiết từ bước trước:

```
def get_refactor_user_prompt(assistant_output: str) -> str:
    code: str = re.sub(
        r"(.*)```\python(.*)```(.*)",
        r"\2",
        assistant_output,
        flags=re.DOTALL
    ).strip()

    user_prompt: str = f"""
    OLD: {{{{{{ {code} }}}}}}
    REFACTORED:
    """

    return user_prompt
```

Hình 27: Giải pháp Selective History

Kết hợp với prompt mới cho nhiệm vụ tiếp theo

```
prompt_1: str = f"{SURROUND} {SINGLE_TASK}"
prompt_2: str = "You are provided with a Python code enclosed in {{{
FUNCTION }}}. Your task is to add type hints to all variables."
prompt_3: str = "You are provided with a Python code enclosed in {{{
FUNCTION }}}. Your task is to include Google Style docstring."
```

Hình 28: Giải pháp Selective History (v2)

Gọi API với selective history

```
for prompt, func in prompts.items():
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": func(next_input)},
        ],
    )
    next_input = completion.choices[0].message.content
```

Hình 29: Giải pháp Selective History (v3)

Đánh giá

1. Chất lượng output tương đương cách naive
2. Nhưng:

- Token giảm mạnh (từ 1,253 → 296, giảm hơn 75%)
- Prompt ngắn gọn, tập trung hơn
- Giảm chi phí và tăng hiệu suất

Selective History là một kỹ thuật quan trọng khi sử dụng OpenAI API trong các bài toán chaining nhiều bước. Bằng cách chỉ giữ lại thông tin cần thiết, phương pháp này giúp:

- Giảm chi phí xử lý
- Tăng hiệu quả của mô hình
- Duy trì chất lượng output

Kỹ thuật này đặc biệt hữu ích trong các hệ thống thực tế, nơi các chuỗi prompt có thể trở nên dài và phức tạp trong suốt vòng đời phát triển phần mềm (SDLC).

9.7.4 Áp dụng Prompt Chaining với GitHub Copilot

Bên cạnh việc sử dụng Chain of Thought (CoT) để xây dựng logic ban đầu, GitHub Copilot cũng hỗ trợ hiệu quả kỹ thuật Prompt Chaining nhằm mở rộng và hoàn thiện mã nguồn.

Đối với các tác vụ vượt ngoài phân cài đặt cơ bản như:

- Thêm docstring
- Kiểm tra đầu vào (input validation)
- Ghi log
- Viết test

việc chia nhỏ thành nhiều bước (chaining) sẽ giúp Copilot xử lý tốt hơn và tạo ra code chất lượng cao hơn.

Một quy trình chaining điển hình với Copilot có thể bao gồm:

- Bước 1: Cài đặt hàm bằng CoT
- Bước 2: Thêm docstring cho tất cả các hàm
- Bước 3: Bổ sung input validation

Ví dụ minh họa

Sau khi đã có implementation ban đầu của hàm `get_average_return`, ta tiếp tục với bước chaining thứ hai.

Prompt thêm docstring

Trong VS Code hoặc PyCharm, sử dụng Copilot Chat với prompt:

Add Google Style docstring to all functions.

Kết quả

Copilot sẽ:

- Tự động thêm docstring cho tất cả các hàm
- Sử dụng đúng chuẩn Google Style
- Mô tả chính xác chức năng của từng hàm

Ví

dụ:

```
def get_gross_returns(net_returns: Dict[str, float]) -> np.ndarray:
    """Converts net returns to gross returns.

    Args:
        net_returns (Dict[str, float]): A dictionary of net returns.

    Returns:
        np.ndarray: Array of gross returns.
    """
```

Hình 30: Áp dụng Prompt Chaining với GitHub Copilot

Công cụ hỗ trợ

VS Code:

- Sử dụng Copilot Chat → chế độ Edit
- Áp dụng trực tiếp thay đổi vào file

PyCharm:

- Mở Copilot Chat (Ctrl + Shift + C)
- Nhập prompt và nhận gợi ý chỉnh sửa

Ưu điểm:

- Dễ sử dụng, tích hợp trực tiếp trong IDE
- Không cần viết lại toàn bộ code
- Có thể chỉnh sửa từng phần theo nhu cầu

Hiệu quả:

- Docstring được tạo chính xác
- Phù hợp với chuẩn coding
- Giảm thời gian viết tài liệu code

Prompt Chaining với GitHub Copilot là một phương pháp hiệu quả để nâng cấp code từ mức cơ bản lên mức hoàn chỉnh. Khi kết hợp với CoT:

- CoT giúp xây dựng logic ban đầu
- Chaining giúp hoàn thiện và tối ưu code

Phương pháp này không chỉ cải thiện chất lượng mã nguồn mà còn hỗ trợ tốt trong các giai đoạn khác của vòng đời phát triển phần mềm (SDLC), như kiểm thử, tối ưu hiệu năng và tài liệu hóa.

9.8 Tổng kết

Chương này trình bày cách sử dụng hiệu quả Chain of Thought (CoT) và Prompt Chaining để cải thiện chất lượng code sinh ra từ các mô hình LLM.

CoT giúp chia nhỏ bài toán thành các bước logic rõ ràng, giúp mô hình xử lý chính xác hơn các bài toán phức tạp. Trong khi đó, Prompt Chaining cho phép cải thiện code theo từng bước, từ một phiên bản đúng chức năng đến phiên bản hoàn thiện hơn (thêm type hints, docstring,...).

Ngoài ra, kỹ thuật Selective History khi sử dụng OpenAI API giúp giảm số lượng token và tăng hiệu quả xử lý bằng cách chỉ giữ lại thông tin cần thiết.

Nhìn chung, việc kết hợp CoT và Prompt Chaining giúp nâng cao chất lượng code và tối ưu quá trình làm việc với AI.

CHƯƠNG 10: REFACTORING CODE VỚI GENAI

10.1. Giới thiệu

Trong phát triển phần mềm, mã nguồn (source code) không chỉ đóng vai trò là tập hợp các chỉ dẫn cho máy tính thực thi mà còn là phương tiện giao tiếp giữa các lập trình viên. Một đoạn code sau khi được viết ra sẽ được đọc lại nhiều lần trong tương lai nhằm phục vụ cho việc bảo trì, mở rộng và tích hợp hệ thống. Vì vậy, chất lượng code – đặc biệt là tính rõ ràng và dễ hiểu – là yếu tố then chốt.

Chương này tập trung vào kỹ thuật refactoring (tái cấu trúc code) với sự hỗ trợ của Generative AI (GenAI), nhằm cải thiện chất lượng code mà không làm thay đổi chức năng ban đầu. Nội dung chương nhấn mạnh cách sử dụng các công cụ như ChatGPT, GitHub Copilot và OpenAI API để tối ưu hóa code về mặt cấu trúc, khả năng đọc và hiệu năng.

10.2. Khái niệm Refactoring

10.2.1. Định nghĩa kỹ thuật

Refactoring được định nghĩa là quá trình chỉnh sửa, tinh chỉnh hệ thống mã nguồn hiện tại nhằm cải thiện cấu trúc bên trong mà không làm thay đổi hành vi và chức năng vốn có của chương trình. Đây là một kỹ thuật có kỷ luật, tập trung vào việc chuyển đổi mã nguồn từ trạng thái "hoạt động được" sang trạng thái "hoạt động tốt và dễ quản lý".

10.2.2. Các mục tiêu trọng tâm

Quá trình Refactoring được thực hiện nhằm đạt được ba giá trị cốt lõi sau:

- Cải thiện khả năng đọc (Readability): Chuyển đổi mã nguồn từ ngôn ngữ máy hoặc các logic phức tạp sang dạng ngôn ngữ gần gũi với con người. Việc mã nguồn dễ đọc giúp giảm thời gian tiếp nhận (onboarding) cho nhân sự mới và hạn chế sai sót trong quá trình đọc hiểu logic.
- Tối ưu hóa cấu trúc (Structure): Áp dụng các mẫu thiết kế (Design Patterns) để loại bỏ sự phụ thuộc quá mức giữa các thành phần (low coupling) và tăng tính đóng gói (high cohesion).
- Nâng cao hiệu năng (Performance): Rà soát và thay thế các thuật toán cũ, cấu trúc dữ liệu kém hiệu quả bằng các giải pháp tối ưu hơn về mặt thời gian thực thi hoặc bộ nhớ.

10.2.3. Tầm quan trọng trong vòng đời phần mềm

Dựa trên các nghiên cứu về kỹ thuật phần mềm, cấu trúc chi phí của một dự án thường có sự chênh lệch lớn giữa các giai đoạn:

"Khoảng 75% chi phí phát triển phần mềm tập trung vào giai đoạn Bảo trì (Maintenance)."

Do đó, Refactoring không chỉ là công việc của lập trình viên mà còn là một chiến lược kinh tế. Việc thực hiện Refactoring thường xuyên giúp:

- Giảm nợ kỹ thuật (Technical Debt): Ngăn chặn việc tích tụ các đoạn code "rác" hoặc các giải pháp tạm thời gây khó khăn cho việc mở rộng sau này.
- Tăng tốc độ phát triển: Code sạch giúp việc phát triển tính năng mới nhanh hơn và ít lỗi hơn.
- Tối ưu hóa chi phí: Giảm thiểu rủi ro phải "đập đi xây lại" toàn bộ hệ thống do mã nguồn quá rối rắm không thể bảo trì.

10.3. Ví dụ minh họa: Hàm tính khoảng cách

Để làm rõ giá trị của Refactoring, chương này phân tích một trường hợp điển hình trong xử lý dữ liệu: Xây dựng hàm tính toán khoảng cách giữa hai ma trận dựa trên hai phương thức phổ biến là Manhattan (L1) và Euclidean (L2).

```
import numpy as np
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route("/distances", methods=["POST"])
def calculate_distance():
    data = request.get_json()
    dist_type = data.get("distance")
    if dist_type == "L1":
        print("Info: computing L1 distance...")
        a = data.get("df1")
        b = data.get("df2")
        dist = np.sum(np.abs(a - b))
        return jsonify({"distance": dist})
    elif dist_type == "L2":
        print("Info: computing L2 distance...")
        a = data.get("df1")
        b = data.get("df2")
        dist_2 = 0
        for i in range(len(a)):
            for j in range(len(a[i])):
                dist_2 += (a[i][j] - b[i][j]) ** 2
        dist = dist_2 ** 0.5
        return jsonify({"distance": dist})

if __name__ == "__main__":
    app.run(debug=True)
```

10.3.1. Phân tích phiên bản mã nguồn ban đầu

Trong phiên bản chưa được refactor, các hàm thường được viết theo lối tư duy tuyến tính, dẫn đến nhiều hạn chế về kỹ thuật:

- Sử dụng vòng lặp lồng nhau (Nested Loops): Việc sử dụng quá nhiều vòng lặp for lồng nhau để duyệt qua các phần tử ma trận khiến độ phức tạp thuật toán tăng cao, gây suy giảm hiệu năng nghiêm trọng khi kích thước dữ liệu lớn.
- Lặp lại logic (Duplicate Logic): Cấu trúc tính toán cho L1 và L2 có nhiều điểm tương đồng nhưng lại được viết tách biệt, dẫn đến dư thừa mã nguồn.
- Vi phạm nguyên tắc "Hard-coded": Các tham số và phương thức tính toán bị đóng cứng trong hàm, khiến việc thay đổi hoặc bảo trì trở nên khó khăn.

10.3.2. Nhận diện "Code Smell" (Dấu hiệu mã nguồn xấu)

Dựa trên phiên bản ban đầu, chúng ta nhận thấy các dấu hiệu đặc trưng của một hệ thống cần được tái cấu trúc ngay lập tức:

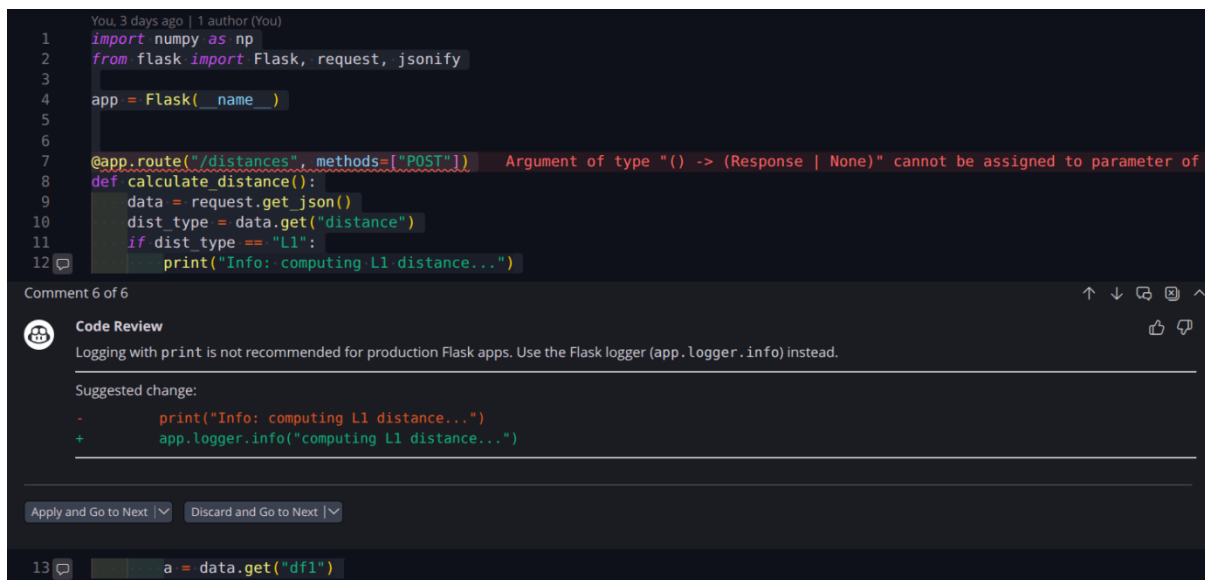
- Rigidity (Tính cứng nhắc): Khó thay đổi vì một thay đổi nhỏ có thể ảnh hưởng đến nhiều vị trí lặp code.
- Fragility (Tính dễ vỡ): Code dễ phát sinh lỗi logic khi người sau cố gắng chỉnh sửa các vòng lặp phức tạp.
- Low Reusability (Tính tái sử dụng thấp): Logic tính toán bị trộn lẫn với logic duyệt mảng, không thể tách rời để dùng cho các mục tiêu khác.

10.3.3. Định hướng Refactoring

Để giải quyết các vấn đề trên, quy trình refactor sẽ tập trung vào:

- Vectorization (Vector hóa): Thay thế vòng lặp bằng các thư viện tối ưu (như NumPy trong Python) để tận dụng sức mạnh tính toán song song.
- Abstraction (Trừu tượng hóa): Tách biệt phần "duyet dữ liệu" và phần "công thức tính toán" thành các module riêng biệt.

Parameterization: Cho phép người dùng lựa chọn loại khoảng cách thông qua tham số, thay vì viết nhiều hàm tương tự nhau.



Hình 31: Định hướng Refactoring

10.4. Các loại Refactoring

Để quá trình cải tiến mã nguồn đạt hiệu quả cao nhất, các kỹ thuật Refactoring thường được phân chia thành ba nhóm chiến lược chính. Mỗi nhóm tập trung vào một khía cạnh cụ thể của chất lượng phần mềm.

10.4.1. Cải thiện khả năng đọc (Readability)

Đây là bước cơ bản nhất, tập trung vào việc làm cho mã nguồn trở nên "trong suốt" đối với người lập trình.

- **Đổi tên biến rõ nghĩa (Renaming):** Thay thế các tên biến vô nghĩa (như a, b, temp) bằng các danh từ mô tả đúng bản chất dữ liệu (như user_balance, matrix_diagonal).
- **Loại bỏ code dư thừa (Dead Code Removal):** Xóa bỏ các đoạn mã không còn được sử dụng, các biến khởi tạo nhưng không dùng đến để giảm nhiễu thông tin.
- **Tránh sử dụng "Magic Numbers":** Thay thế các con số cụ thể xuất hiện bất ngờ trong code bằng các hằng số có tên gọi rõ ràng (ví dụ: thay 3.14 bằng PI_CONSTANT).
- **Bổ sung Type Hints:** Sử dụng chú thích kiểu dữ liệu (trong các ngôn ngữ như Python, TypeScript) để giúp các công cụ lập trình (IDE) hỗ trợ bắt lỗi sớm và tăng khả năng đọc hiểu logic.

10.4.2. Thay đổi cấu trúc (Structure)

Nhóm này tập trung vào việc tổ chức lại "bản vẽ" của chương trình để tăng tính linh hoạt và dễ bảo trì.

- Tách hàm (Extract Method): Chia nhỏ các hàm quá dài (Long Method) thành các hàm con chuyên biệt, mỗi hàm chỉ thực hiện một nhiệm vụ duy nhất (Single Responsibility).
- Áp dụng nguyên tắc DRY (Don't Repeat Yourself): Nhận diện các đoạn mã lặp lại và đóng gói chúng vào một hàm hoặc lớp dùng chung, giúp việc chỉnh sửa sau này chỉ cần thực hiện tại một nơi.
- Tổ chức lại logic chương trình: Chuyển đổi các cấu trúc if-else phức tạp thành các cấu trúc điều khiển minh bạch hơn hoặc sử dụng các mẫu thiết kế (Design Patterns) phù hợp.

10.4.3. Tối ưu hiệu năng (Performance)

Khi cấu trúc đã sạch sẽ, việc tối ưu tốc độ và tài nguyên sẽ trở nên dễ dàng và chính xác hơn.

- Thay đổi thuật toán: Thay thế các thuật toán có độ phức tạp lớn bằng các thuật toán tối ưu hơn (ví dụ: chuyển từ tìm kiếm tuần tự sang tìm kiếm nhị phân).
- Sử dụng Vectorization thay cho vòng lặp: Trong xử lý dữ liệu, việc thay thế vòng lặp for bằng các phép toán vector giúp tận dụng tối đa sức mạnh của CPU/GPU.
- Tận dụng thư viện tối ưu (như NumPy): Sử dụng các thư viện chuyên dụng đã được viết bằng ngôn ngữ bậc thấp (C/C++) để xử lý các tác vụ tính toán nặng, mang lại tốc độ vượt trội so với mã nguồn thuần.

10.5. Hạn chế của công cụ IDE truyền thống

Trong kỷ nguyên công nghệ hiện đại, việc Refactoring không còn là một quá trình thủ công hoàn toàn. Tuy nhiên, sự khác biệt giữa các công cụ truyền thống và công nghệ mới đang tạo ra những bước ngoặt lớn.

10.5.1. Khả năng của các IDE phổ biến (VS Code, PyCharm, v.v.)

Các môi trường phát triển tích hợp (IDE) hiện nay cung cấp các bộ công cụ "Refactor" tự động rất mạnh mẽ, giúp giảm thiểu sai sót do con người:

- Rename (Đổi tên): Tự động cập nhật tên biến, hàm hoặc lớp ở tất cả các tệp tin liên quan trong dự án, đảm bảo tính nhất quán.
- Extract Method (Trích xuất hàm): Tự động tách một đoạn mã được chọn thành một hàm riêng biệt và thiết lập các tham số truyền vào phù hợp.
- Auto-Format (Định dạng mã): Căn chỉnh lề, khoảng cách và cấu trúc code theo các chuẩn mực như PEP 8 (Python) hay Prettier (JavaScript).

10.5.2. Những hạn chế của công cụ truyền thống

Mặc dù hữu ích, các IDE truyền thống vẫn vận hành dựa trên các quy tắc (rule-based) cứng nhắc, dẫn đến các giới hạn:

- Không xử lý được logic phức tạp: IDE có thể đổi tên biến, nhưng không thể hiểu rằng một cấu trúc if-else lồng nhau có thể được chuyển đổi thành một cấu trúc match-case hoặc strategy pattern để gọn gàng hơn.
- Hạn chế với mã không liên tục: Các công cụ này khó có thể gom nhóm các đoạn mã nằm rải rác ở nhiều vị trí khác nhau để tái cấu trúc thành một logic thống nhất.
- Thiếu sự thấu hiểu ngữ cảnh (Contextual Blindness): IDE không thực sự hiểu "ý nghĩa" của đoạn code. Nó không biết đoạn mã đó đang thực hiện nhiệm vụ gì trong tổng thể dự án để đưa ra những đề xuất cải tiến về mặt kiến trúc.

10.5.3. Sự trỗi dậy của GenAI trong Refactoring

Sự xuất hiện của Generative AI (như GitHub Copilot, ChatGPT) đã lấp đầy những khoảng trống mà IDE truyền thống để lại. GenAI có khả năng:

- Phân tích ngữ cảnh sâu: Hiểu mục đích của đoạn mã để đề xuất cách tổ chức lại logic một cách thông minh.
- Đề xuất các kỹ thuật nâng cao: Gợi ý áp dụng các mẫu thiết kế (Design Patterns) phù hợp mà IDE thông thường không thể nhận diện.
- Hỗ trợ Vectorization: Tự động nhận diện các vòng lặp kém hiệu quả và chuyển đổi chúng sang dạng Vectorization (sử dụng thư viện như NumPy) để tối ưu hiệu năng.

10.6. Ứng dụng GenAI trong Refactoring

Sự xuất hiện của các mô hình ngôn ngữ lớn (LLMs) đã chuyển đổi Refactoring từ một công việc thủ công tiêu tốn thời gian thành một quá trình cộng tác thông minh. Việc ứng dụng đúng kỹ thuật Prompt Engineering là chìa khóa để đảm bảo mã nguồn mới vừa sạch, vừa chính xác.

10.6.1. Kỹ thuật Chuỗi tư duy (Chain-of-Thought - CoT)

CoT là phương pháp hướng dẫn AI suy luận từng bước thay vì yêu cầu kết quả cuối cùng ngay lập tức. Đối với các hàm phức tạp như tính toán ma trận, việc chia nhỏ logic giúp AI tránh được các lỗi suy luận bậc cao.

Quy trình cung cấp cho AI: Thay vì ra lệnh "Refactor hàm này", ta cung cấp các bước xử lý chính:

Giai đoạn Parse: Phân tích và chuẩn hóa dữ liệu đầu vào.

- Giai đoạn Phân loại: Xác định loại khoảng cách (Manhattan hay Euclidean).
- Giai đoạn Thực thi: Gọi các hàm chuyên biệt đã tối ưu.

Kết quả: AI đóng vai trò "người thực hiện chi tiết" dựa trên "bản thiết kế logic" của lập trình viên, giúp mã nguồn bám sát ý đồ thiết kế ban đầu.

10.6.2. Refactoring tương tác với ChatGPT và Copilot

Trong môi trường làm việc trực tiếp, các công cụ như ChatGPT hoặc GitHub Copilot hoạt động hiệu quả nhất khi được cung cấp đầy đủ ngữ cảnh thông qua phương pháp đối chiếu:

1. Nguyên tắc "Cũ & Mới": Cung cấp cả đoạn Code cũ (Reference) để AI hiểu hành vi hiện tại và mẫu Cấu trúc mới (Target Structure) mà bạn muốn hướng tới.
2. Lợi ích:
 - Bảo toàn logic: Đảm bảo AI không vô tình thay đổi các quy tắc nghiệp vụ quan trọng.
 - Tùy biến phong cách: Ép AI tuân thủ các quy chuẩn riêng của dự án thay vì sử dụng code mẫu chung chung trên mạng.

10.6.3. Tự động hóa quy mô lớn với OpenAI API

Khi đối mặt với dự án có hàng nghìn dòng code hoặc cần Refactor đồng bộ trên nhiều tệp tin, việc sử dụng giao diện Chat là không khả thi. OpenAI API cho phép tích hợp AI vào quy trình phát triển tự động.

- Ưu điểm: Khả năng xử lý hàng loạt, tích hợp trực tiếp vào pipeline CI/CD để tự động đề xuất bản vá (Pull Request) cho các đoạn mã chưa tối ưu.
- Thách thức: Thiết kế Prompt cho API đòi hỏi tính kỹ thuật cao hơn (System Prompt) để kiểm soát định dạng đầu ra (ví dụ: chỉ trả về mã nguồn dạng JSON hoặc Markdown mà không có lời giải thích thừa).

10.7. Refactoring hiệu năng với GenAI

Trong các dự án xử lý dữ liệu lớn, hiệu năng không chỉ là một lựa chọn mà là yêu cầu sống còn. GenAI đóng vai trò như một "chuyên gia tối ưu" giúp phát hiện và chuyển đổi các cấu trúc mã nguồn cũ kỹ sang các dạng thức hiện đại.

10.7.1. Chuyển đổi tư duy: Từ Scalar sang Vector

Một trong những cải tiến quan trọng nhất mà GenAI thường đề xuất khi refactor code xử lý mảng là Vectorization (Vector hóa). Thay vì xử lý từng phần tử riêng lẻ bằng vòng lặp, chúng ta thực hiện phép toán trên toàn bộ mảng cùng lúc.

Ví dụ minh họa tính khoảng cách (Euclidean Distance):

Cách tiếp cận cũ (Legacy Code): Sử dụng các vòng lặp lồng nhau, khiến CPU phải thực hiện hàng triệu lệnh rẽ nhánh liên tiếp.

```
for i in range(n):
    for j in range(m):
        dist += (a[i][j] - b[i][j]) ** 2
result = math.sqrt(dist)
```


Cách tiếp cận mới (Refactored với GenAI): Sử dụng thư viện NumPy để thực hiện phép toán song song.

```
result = np.sqrt(np.sum((a - b) ** 2))
```

10.7.2. Lợi ích của việc Refactor hiệu năng qua GenAI

Khi yêu cầu GenAI (như ChatGPT hay Copilot) tối ưu hóa các đoạn mã này, hệ thống sẽ mang lại ba giá trị vượt trội:

- Giảm thời gian thực thi (Runtime): Loại bỏ chi phí quản lý vòng lặp của ngôn ngữ bậc cao (như Python). Các phép toán vector được đẩy xuống tầng thấp hơn (C/C++ hoặc Fortran) để xử lý.
- Tận dụng tối ưu phần cứng: Tận dụng tập lệnh SIMD (Single Instruction, Multiple Data) của CPU hiện đại, cho phép xử lý nhiều luồng dữ liệu trong một chu kỳ xung nhịp duy nhất.
- Mã nguồn ngắn gọn (Clean Code): Code giảm từ 5-6 dòng xuống còn 1 dòng duy nhất, giúp tăng khả năng đọc hiểu và giảm thiểu các lỗi "off-by-one" thường gặp trong vòng lặp.

10.7.3. Vai trò của GenAI trong quy trình này

GenAI không chỉ đơn giản là viết lại code, nó giúp lập trình viên:

- Nhận diện cơ hội: Tự động phát hiện các vòng lặp lồng nhau có thể chuyển đổi sang NumPy.
- Đề xuất thư viện phù hợp: Gợi ý sử dụng NumPy, Pandas hoặc PyTorch tùy theo ngữ cảnh dự án.
- Kiểm chứng: Hỗ trợ viết các đoạn mã so sánh thời gian chạy (Benchmark) để chứng minh hiệu quả của việc Refactor.

10.8. So sánh các công cụ GenAI

Công cụ	Ưu điểm
GitHub Copilot	Nhanh, tích hợp trực tiếp IDE
ChatGPT	Hiểu logic tốt, dễ dùng
OpenAI API	Tự động hóa quy mô lớn

Bảng 3: Bảng so sánh các công cụ GenAI

10.9. Kết luận

Refactoring không còn là một lựa chọn "có thì tốt", mà đã trở thành một phần tất yếu để duy trì sự sống còn và khả năng cạnh tranh của các hệ thống phần mềm hiện đại. Việc thấu hiểu và áp dụng đúng công cụ trong giai đoạn này sẽ quyết định tính bền vững của dự án.

10.9.1. Sự chuyển dịch về công cụ hỗ trợ

Báo cáo đã chỉ ra sự khác biệt rõ rệt giữa hai thế hệ công cụ:

- IDE truyền thống: Thực hiện tốt các "tiểu phẫu" như đổi tên, trích xuất hàm hay định dạng mã. Đây là nền tảng vững chắc cho các thao tác cơ bản hàng ngày.
- GenAI (Trí tuệ nhân tạo tạo sinh): Mang lại một bước nhảy vọt về khả năng thấu hiểu ngữ cảnh. AI không chỉ sửa lỗi cú pháp mà còn tham gia vào việc tái cấu trúc logic phức tạp và tối ưu hiệu năng ở cấp độ thuật toán (như Vectorization).

10.9.2. Triết lý phát triển bền vững

Dù công nghệ hỗ trợ có tiến bộ đến đâu, cốt lõi của phát triển phần mềm vẫn nằm ở tư duy con người:

- Viết code sạch ngay từ đầu: Đây vẫn là giải pháp kinh tế và kỹ thuật tối ưu nhất. Refactoring là quá trình cải tiến, không nên là cái cớ cho việc viết code cầu thả.
- Vai trò của lập trình viên: GenAI là một trợ thủ đắc lực giúp tăng năng suất lao động, nhưng không thể thay thế hoàn toàn kỹ năng lập trình và khả năng ra quyết định chiến lược của con người.

10.9.3. Thông điệp cuối cùng

Đầu tư vào Refactoring, đặc biệt là với sự hỗ trợ của các kỹ thuật như Chain-of-Thought hay OpenAI API, sẽ giúp doanh nghiệp giảm thiểu gánh nặng từ 75% chi phí bảo trì, đồng thời tạo ra một môi trường làm việc chuyên nghiệp, nơi mã nguồn luôn sẵn sàng cho những sự thay đổi của tương lai.

CHƯƠNG 11: FINE-TUNING MÔ HÌNH VỚI OPENAI

11.1. Giới thiệu

Trong bối cảnh các mô hình ngôn ngữ lớn (LLMs) ngày càng được ứng dụng rộng rãi trong phát triển phần mềm, việc kiểm soát hành vi và chất lượng đầu ra của mô hình trở thành một yêu cầu quan trọng. Chapter 11 tập trung vào kỹ thuật fine-tuning, một phương pháp cho phép tùy biến mô hình để phục vụ các tác vụ chuyên biệt.

Khác với các kỹ thuật như prompt engineering hay few-shot learning, fine-tuning tác động trực tiếp vào trọng số (weights) của mô hình, giúp mô hình học được hành vi mong muốn một cách ổn định và nhất quán hơn.

11.2. Tổng quan về Fine-tuning

Trong khi Prompt Engineering giúp chúng ta giao tiếp với AI, thì **Fine-tuning** (Tinh chỉnh) là quá trình can thiệp sâu hơn vào "tư duy" của mô hình để đạt được độ chính xác tuyệt đối cho các tác vụ chuyên biệt.

11.2.1. Khái niệm và Bản chất Kỹ thuật

Fine-tuning là quá trình huấn luyện bổ sung một mô hình đã được đào tạo trước (Pre-trained model) trên một tập dữ liệu nhỏ, có cấu trúc và mang tính đặc thù cao.

Thay vì bắt đầu từ con số không, chúng ta tận dụng tri thức không lồ mà mô hình đã có, sau đó:

- Điều chỉnh hành vi: Giúp mô hình hiểu được phong cách phản hồi, thuật ngữ chuyên ngành hoặc quy trình làm việc riêng biệt của một tổ chức.
- Tối ưu hóa nhiệm vụ: Biến một mô hình đa năng thành một "chuyên gia" trong một lĩnh vực hẹp (ví dụ: chuyên gia Refactor code Python, chuyên gia viết Unit Test cho hệ thống ngân hàng).

11.2.2. Mục tiêu chiến lược của Fine-tuning

Việc áp dụng Fine-tuning mang lại ba lợi ích cốt lõi giúp vượt qua các giới hạn của Prompt Engineering thông thường:

- Tách biệt "Logic hướng dẫn" khỏi Prompt: Trong các hệ thống thông thường, bạn phải viết một System Prompt rất dài để dạy AI cách hành xử. Với Fine-tuning, các quy tắc này được "đúc" trực tiếp vào trọng số (weights) của mô hình. AI tự hiểu cách làm việc mà không cần nhắc lại quy trình.
- Tối ưu hóa tài nguyên (Giảm độ dài Prompt): Bằng cách giảm bớt các chỉ dẫn lặp đi lặp lại và các ví dụ (few-shot examples) trong prompt, chúng ta tiết kiệm được đáng kể lượng Token. Điều này trực tiếp làm giảm chi phí vận hành API và giảm độ trễ (latency) khi xử lý dữ liệu.

- Tăng tính nhất quán của Output: Fine-tuning giúp triệt tiêu sự ngẫu nhiên không mong muốn. Mô hình sẽ luôn trả về kết quả với định dạng, cấu trúc và văn phong thống nhất 100%, đáp ứng các tiêu chuẩn khắt khe của hệ thống phần mềm doanh nghiệp.

11.2.3. So sánh nhanh: Prompt Engineering vs. Fine-tuning

Đặc điểm	Prompt Engineering	Fine-tuning
Chi phí thiết lập	Thấp (chỉ cần viết prompt)	Cao (cần dữ liệu và GPU)
Độ chính xác	Phụ thuộc vào ngữ cảnh	Rất cao và ổn định
Khả năng cập nhật	Tức thời	Cần huấn luyện lại
Ứng dụng tối ưu	Thử nghiệm, tác vụ thay đổi	Hệ thống sản xuất (Production), tác vụ lặp lại

Bảng 4: Bảng so sánh nhanh giữa Prompt Engineering và Fine-Tuning

11.3. So sánh Fine-tuning và Few-shot Learning

Tiêu chí	Few-shot Learning	Fine-tuning
Cách hoạt động	Ví dụ trong prompt	Huấn luyện lại model
Tính linh hoạt	Cao	Thấp hơn
Chi phí runtime	Cao (prompt dài)	Thấp hơn
Khả năng mở rộng	Hạn chế	Tốt

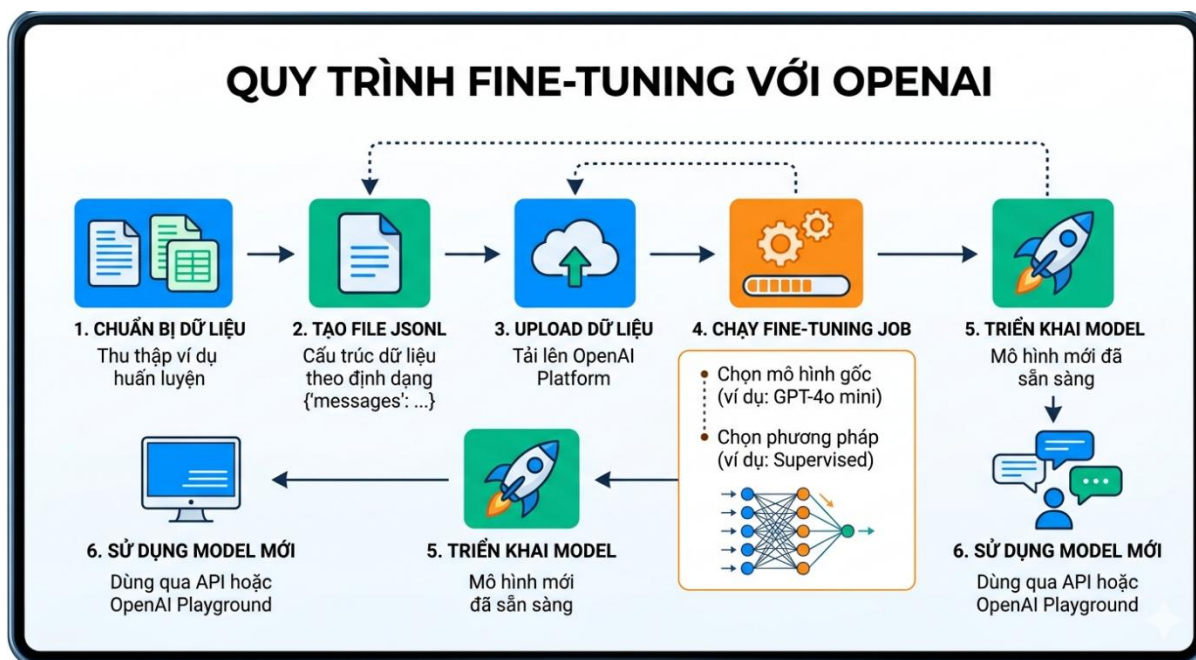
Bảng 5: Bảng so sánh Fine-Tuning và Few-shot Learning

Nhận xét:

Few-shot learning phù hợp cho các bài toán nhỏ, trong khi fine-tuning phù hợp cho các hệ thống lớn cần tính ổn định cao.

10.4. Quy trình Fine-tuning

Việc tinh chỉnh mô hình không chỉ là một thao tác kỹ thuật, mà là một quy trình có kỷ luật gồm 6 bước cốt lõi. Dưới đây là sơ đồ trực quan hóa toàn bộ dòng chảy dữ liệu và công việc:



Hình 32: Quy trình Fine-Tuning với OpenAI

Mô tả chi tiết các bước trong sơ đồ:

1. Chuẩn bị Dữ liệu (Data Preparation): Đây là bước quyết định sự thành bại. Chúng ta cần thu thập một tập hợp các ví dụ (vài chục đến vài trăm) mô tả chính xác cách chúng ta muốn mô hình hành xử cho một nhiệm vụ cụ thể.
2. Tạo File JSONL (JSONL File Creation): Mô hình OpenAI đòi hỏi dữ liệu phải được định dạng theo chuẩn JSON Lines (JSONL). Mỗi dòng trong tệp tin là một đối tượng chứa trọn vẹn System Prompt, User Input, và Assistant Output mong muốn.
3. Upload Dữ liệu (Upload to OpenAI Platform): Sử dụng OpenAI API hoặc giao diện trên nền tảng (Platform) để tải tệp JSONL đã được định dạng chuẩn lên máy chủ an toàn của OpenAI. Tệp này sẽ được gán một file-id duy nhất.
4. Chạy Fine-tuning Job (Run Fine-Tuning Job): Khởi tạo quá trình huấn luyện bằng cách cung cấp file-id từ bước 3 và cấu hình các tham số quan trọng:
 - Mô hình gốc (Base Model): Chọn mô hình bạn muốn tinh chỉnh (ví dụ: gpt-4o-mini).

- Phương pháp: Thông thường sử dụng phương pháp Học có giám sát (Supervised Learning).
5. Triển khai Model (Deploy Model): Sau khi quá trình huấn luyện hoàn tất thành công, mô hình mới sẽ được đăng ký vào tài khoản OpenAI của bạn. Mô hình mới này sẽ có một tên định danh duy nhất (ví dụ: ft:gpt-4o-mini-0123:your-org:custom-name).
 6. Sử dụng Model Mới (Use New Model): Mô hình đã được tinh chỉnh giờ đây sẵn sàng để sử dụng. Bạn có thể gọi nó trực tiếp qua API trong ứng dụng sản xuất, hoặc kiểm thử nhanh chóng trong OpenAI Playground.

11.5. Thiết kế dữ liệu huấn luyện

Để mô hình có thể học hỏi từ các ví dụ, dữ liệu cần được đóng gói dưới định dạng JSONL (JSON Lines). Đây là định dạng mà mỗi dòng trong tệp tin là một đối tượng JSON hoàn chỉnh, cho phép hệ thống xử lý song song và tối ưu bộ nhớ.

11.5.1. Cấu trúc một bản ghi mẫu

Một đối tượng dữ liệu chuẩn cho OpenAI Fine-tuning sẽ bao gồm một mảng **messages** mô phỏng lại một cuộc hội thoại thực tế:

```
{
  "messages": [
    { "role": "system", "content": "Bạn là một chuyên gia Refactoring mã nguồn Python." },
    { "role": "user", "content": "Hãy tối ưu vòng lặp tính tổng bình phương này." },
    { "role": "assistant", "content": "Bạn nên sử dụng np.dot(a, a) để đạt hiệu năng cao nhất.", "weight": 1 }
  ]
}
```

11.5.2. Ý nghĩa và vai trò của các thành phần

1. Việc hiểu rõ từng thành phần giúp lập trình viên kiểm soát chính xác hành vi của mô hình sau khi huấn luyện:
2. **system** (Hệ thống): Thiết lập "công tắc" ngữ cảnh. Đây là nơi bạn định nghĩa vai trò, các quy tắc nghiêm ngặt hoặc phong cách phản hồi mà mô hình phải tuân theo xuyên suốt dự án.
3. **user** (Người dùng): Đại diện cho dữ liệu đầu vào thực tế (Input). Trong ngữ cảnh Refactoring, đây thường là đoạn mã nguồn cũ (Legacy Code) kèm theo yêu cầu cụ thể.
4. **assistant** (Trợ lý): Câu trả lời mẫu lý tưởng mà bạn muốn mô hình tạo ra (Target Output). Đây chính là "đáp án" để mô hình học tập.
5. **weight** (Trọng số): Đây là tham số nâng cao để điều chỉnh quá trình học:

- 1 (Mặc định): Thông báo cho mô hình rằng đây là kết quả đúng, cần được học và mô phỏng theo.
- 0 (Loại trừ): Sử dụng khi bạn muốn cung cấp các ví dụ về kết quả sai hoặc không mong muốn để mô hình nhận diện nhưng không bắt buộc (giúp mô hình phân biệt tốt hơn giữa code sạch và code xấu).

11.6. Contrastive Learning trong Fine-tuning

Để mô hình không chỉ "bắt buộc" mà còn thực sự "hiểu" được sự khác biệt giữa các tiêu chuẩn mã nguồn, kỹ thuật Contrastive Learning (Học tương phản) được áp dụng như một phương pháp huấn luyện chuyên sâu.

11.6.1. Khái niệm và Bản chất

Contrastive Learning là kỹ thuật huấn luyện bằng cách cung cấp đồng thời cả ví dụ tích cực (hành vi mong muốn) và ví dụ tiêu cực (hành vi cần tránh).

Thay vì chỉ dạy mô hình "đây là kết quả đúng", chúng ta cung cấp thêm ngữ cảnh về "đây là kết quả sai/không phù hợp". Điều này giúp mô hình hình thành một "biên giới quyết định" sắc nét hơn trong không gian vector dữ liệu.

11.6.2. Ứng dụng cụ thể qua Trọng số (Weight)

Trong cấu trúc JSONL của OpenAI, chúng ta sử dụng tham số weight để hiện thực hóa kỹ thuật này. Hãy xem xét ví dụ về việc ép mô hình chỉ trả về mã nguồn thuần túy mà không kèm theo lời giải thích dư thừa:

Ví dụ Tiêu cực (Negative Example):

- Assistant Output: "Dưới đây là đoạn code đã tối ưu: `np.sum(a**2)`..."
- Weight: 0
- Mục đích: Thông báo cho mô hình rằng định dạng có lời dẫn giải này là không ưu tiên, không được tính vào hàm mất mát (loss function) của quá trình học.

Ví dụ Tích cực (Positive Example):

- Assistant Output: `np.sum(a**2)`
- Weight: 1
- Mục đích: Khẳng định đây là định dạng chuẩn (chỉ có code), mô hình cần tập trung tối đa để mô phỏng theo.

11.6.3. Lợi ích vượt trội

Việc áp dụng Contrastive Learning mang lại những giá trị thực tế sau:

1. Tăng độ chính xác tuyệt đối (High Precision): Mô hình phân biệt rõ ràng giữa các khái niệm dễ gây nhầm lẫn, từ đó đưa ra phản hồi chính xác hơn trong các tình huống phức tạp.
2. Giảm hành vi không mong muốn (Zero Noise): Triệt tiêu các thói quen xấu của mô hình ngôn ngữ gốc (như nói quá nhiều, giải thích rườm rà hoặc tự ý thêm bớt logic ngoài yêu cầu).
3. Tối ưu hóa Pipeline tự động: Khi output chỉ chứa code (nhờ weight = 1), hệ thống có thể lấy kết quả từ API và thực thi (execute) ngay lập tức mà không cần qua các bước xử lý chuỗi (string parsing) phức tạp.

11.7. Case Study: Sinh code sạch (Clean Code Generation)

Việc Fine-tuning không chỉ là một quá trình kỹ thuật mà là một cuộc "cải cách" về tư duy cho mô hình ngôn ngữ, giúp giải quyết triệt để các rào cản trong quy trình Refactoring tự động.

11.7.1. Những vấn đề tồn đọng của Mô hình mặc định (Base Model)

Mặc dù các mô hình như GPT-4o rất mạnh mẽ, nhưng khi áp dụng vào Refactoring chuyên sâu, chúng thường bộc lộ những hạn chế về mặt "phong cách":

- Giải thích dư thừa (Verbosity): Luôn đi kèm các đoạn hội thoại dẫn nhập hoặc kết luận ("Dưới đây là...", "Hy vọng đoạn code này giúp ích...").
- Lạm dụng Comment: Tự ý thêm các dòng chú thích vào giữa mã nguồn, đôi khi làm rối cấu trúc logic đã được tối ưu.
- Cấu trúc chưa tinh gọn: Xu hướng viết code theo lối mòn của dữ liệu huấn luyện chung, dẫn đến việc mã nguồn vẫn còn dài và thiếu tính module hóa.

11.7.2. Giải pháp thông qua Fine-tuning chuyên biệt

Bằng cách cung cấp các cặp dữ liệu chuẩn trong file JSONL và áp dụng trọng số (weight), chúng ta định hướng lại mô hình theo các tiêu chuẩn nghiêm ngặt:

- Chế độ "Code-Only": Huấn luyện mô hình nhận diện rằng chỉ có mã nguồn trong thẻ Markdown (hoặc plain text) mới là kết quả được chấp nhận.
- Nguyên lý "No-Comment Policy": Loại bỏ hoàn toàn các chú thích không cần thiết, tập trung vào việc làm cho mã nguồn tự thân rõ ràng (Self-documenting code).
- Module hóa tự động: Ép mô hình tuân thủ việc tách các khối logic lớn thành các hàm nhỏ (Helper functions), đảm bảo tính tái sử dụng cao.

11.7.3. Kết quả thực nghiệm

Sau quá trình Fine-tuning, mô hình mang lại những cải thiện vượt trội về chất lượng đầu ra:

Chỉ số	Trước Fine-tuning	Sau Fine-tuning
Độ dài Output	Dài (bao gồm giải thích)	Ngắn gọn (chỉ có code)
Tính cấu trúc	Khối code lớn (Monolithic)	Đa hàm (Modularized)
Khả năng đọc	Trung bình (nhiều nhiễu)	Rất cao (Clean code)
Tự động hóa	Cần Parser để lọc code	Có thể thực thi trực tiếp

Bảng 6: Bảng đánh giá kết quả thực nghiệm trước và sau quá trình Fine-Tuning

11.8. Chi phí Fine-tuning

Một trong những rào cản lớn nhất khi tiếp cận Fine-tuning là quan niệm cho rằng chi phí sẽ rất đắt đỏ. Tuy nhiên, khi phân tích kỹ lưỡng dựa trên dữ liệu thực tế từ OpenAI (ví dụ với model gpt-4o-mini), chúng ta sẽ thấy một bức tranh tài chính tối ưu hơn.

11.8.1. Chi phí Huấn luyện (Training Costs)

Chi phí huấn luyện được tính dựa trên số lượng Token trong tập dữ liệu JSONL và số vòng lặp huấn luyện (epochs).

1. Định mức: Khoảng \$3.00 cho mỗi 1 triệu tokens huấn luyện.
2. Phân tích ví dụ thực tế: Với tập dữ liệu mẫu phục vụ Refactoring hàm tính khoảng cách ma trận (khoảng 50-100 ví dụ chất lượng cao):
 - Tổng lượng dữ liệu ước tính: ~13,000 tokens.
 - Chi phí thực tế: Chỉ khoảng \$0.04 (Bốn xu Mỹ).
3. Kết luận: Chi phí đầu tư ban đầu để "dạy" mô hình là cực kỳ thấp, gần như không đáng kể so với lợi ích mang lại.

11.8.2. Chi phí Vận hành (Inference Costs)

Mặc dù giá đơn vị của mô hình đã Fine-tune thường cao gấp đôi (~2 lần) so với mô hình gốc (Base model), nhưng tổng hóa đơn cuối tháng có thể thấp hơn nhờ các yếu tố sau:

- Prompt ngắn hơn: Do logic hướng dẫn đã được "đúc" vào mô hình, bạn không cần viết System Prompt dài hàng nghìn chữ hay cung cấp các ví dụ Few-shot mỗi khi gọi API.
- Output ngắn gọn: Mô hình đã được huấn luyện để chỉ trả về code, loại bỏ hoàn toàn các đoạn giải thích dư thừa (vốn chiếm nhiều token nhất).

- Tỷ lệ thành công cao: Ít phải gọi lại API (retry) do mô hình đã hiểu đúng ý đồ ngay từ lần đầu tiên.

11.8.3. Bảng so sánh tổng hợp

Thành phần	Model Gốc (Prompt Engineering)	Model Fine-tuned
Giá đơn vị Token	Thấp (\$)	Cao (\$\$)
Độ dài System Prompt	Dài (Tốn token)	Rất ngắn (Tiết kiệm)
Độ dài Output	Trung bình/Dài	Ngắn nhất (Chỉ code)
Tổng chi phí/Request	Phụ thuộc vào độ dài chỉ dẫn	Thường tối ưu hơn 20-40%

Bảng 7: Bảng so sánh Model gốc (Prompt Engineering) và Model Fine-Tuned

11.9. Đánh giá hiệu quả

So sánh giữa model gốc và model fine-tuned:

Tiêu chí	Model gốc	Fine-tuned
Độ dài output	Dài	Ngắn
Có giải thích	Có	Không
Structure	Trung bình	Tốt
Cost tổng	Cao	Tối ưu

Bảng 8: Bảng đánh giá hiệu quả giữa model gốc và model sau Fine-Tuned

11.10. Khi nào nên sử dụng Fine-tuning

Mặc dù Fine-tuning mang lại sức mạnh vượt trội, nhưng đây là một khoản đầu tư về dữ liệu và thời gian. Việc xác định đúng thời điểm triển khai sẽ giúp tối ưu hóa nguồn lực của dự án.

11.10.1. Các kịch bản nên sử dụng Fine-tuning

Tác vụ lặp lại với tần suất lớn: Khi hệ thống của bạn phải xử lý hàng ngàn yêu cầu Refactoring mỗi ngày, việc cắt giảm độ dài prompt thông qua Fine-tuning sẽ tiết kiệm một khoản chi phí cực kỳ lớn và giảm độ trễ (latency).

Yêu cầu tính nhất quán (Consistency) tuyệt đối: Nếu dự án yêu cầu output phải tuân thủ nghiêm ngặt một định dạng duy nhất (ví dụ: chỉ trả về mã nguồn, không lời dẫn, đúng chuẩn linting của công ty) mà Prompt thường xuyên làm sai.

Dự án quy mô lớn (Enterprise Scale): Khi cần đóng gói tri thức chuyên môn của các kỹ sư kỳ cựu vào một "bộ não AI" dùng chung cho toàn bộ tập đoàn, đảm bảo mọi lập trình viên đều nhận được sự hỗ trợ với tiêu chuẩn chất lượng như nhau.

Vượt quá giới hạn Context Window: Khi các ví dụ mẫu (Few-shot) và hướng dẫn quá dài khiến prompt chiếm hết không gian xử lý của mô hình, Fine-tuning là cách duy nhất để "dạy" mô hình mà không làm tổn tài nguyên ngữ cảnh.

11.10.2. Các kịch bản không nên sử dụng Fine-tuning

Đừng vội vàng Fine-tune nếu:

- Tác vụ đơn giản, biến thiên cao: Nếu yêu cầu thay đổi theo từng ngày hoặc chỉ là các tác vụ giải đáp thắc mắc thông thường, Prompt Engineering linh hoạt hơn nhiều.
- Kỹ thuật Few-shot đã đủ hiệu quả: Nếu bạn chỉ cần cung cấp 1-2 ví dụ trong prompt mà mô hình đã trả về kết quả chính xác 95-100%, việc Fine-tuning sẽ trở nên dư thừa và tốn kém không cần thiết.
- Thiếu hụt dữ liệu chuẩn: Fine-tuning với dữ liệu "rác" hoặc dữ liệu sai logic sẽ làm hỏng mô hình. Nếu không có ít nhất 50 ví dụ thực sự chất lượng, hãy tiếp tục tối ưu Prompt.

11.11. Kết luận

Fine-tuning không chỉ đơn thuần là một kỹ thuật huấn luyện bổ sung, mà là cầu nối quan trọng đưa các mô hình ngôn ngữ lớn (LLM) từ dạng "trí tuệ tổng quát" trở thành các "công cụ chuyên biệt" có độ chính xác và hiệu quả kinh tế cao trong các hệ thống thực tế.

11.11.1. Những giá trị cốt lõi mang lại

Thông qua việc huấn luyện mô hình với dữ liệu đặc thù và có định hướng, chúng ta đạt được ba mục tiêu then chốt:

- Kiểm soát hành vi tuyệt đối (Behavioral Control): Loại bỏ sự mơ hồ của AI, ép mô hình tuân thủ các quy tắc nghiêm ngặt về định dạng (như chỉ trả về code) và phong cách lập trình của tổ chức.
- Tối ưu hóa chi phí vận hành (Operational Efficiency): Việc rút ngắn prompt và loại bỏ các ví dụ few-shot giúp giảm đáng kể lượng token tiêu thụ, từ đó hạ thấp hóa đơn API hàng tháng dù đơn giá model có thể cao hơn.
- Nâng cao chất lượng đầu ra (Output Excellence): Kết quả đạt được có tính nhất quán cao, giảm thiểu lỗi logic và loại bỏ hoàn toàn các nội dung rác (noise) không cần thiết.

Dù công nghệ Fine-tuning của các nền tảng như OpenAI rất mạnh mẽ, nhưng chất lượng của bộ dữ liệu (Dataset) mới là yếu tố sống còn:

- Một bộ dữ liệu nhỏ nhưng tinh khiết (chất lượng cao, không lỗi logic) luôn mang lại kết quả tốt hơn một bộ dữ liệu lớn nhưng lộn xộn.
- Việc đầu tư thời gian và công sức vào khâu thiết kế dữ liệu huấn luyện (như áp dụng Contrastive Learning) là khoản đầu tư thông minh nhất trong quy trình này.

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Packt Publishing, *Supercharged Coding with GenAI*. Birmingham, UK: Packt Publishing, 2025.

[2] Packt Publishing, "Supercharged Coding with Gen-AI Repository," 2025. [Online]. Available: <https://github.com/PacktPublishing/Supercharged-Coding-with-Gen-AI>.